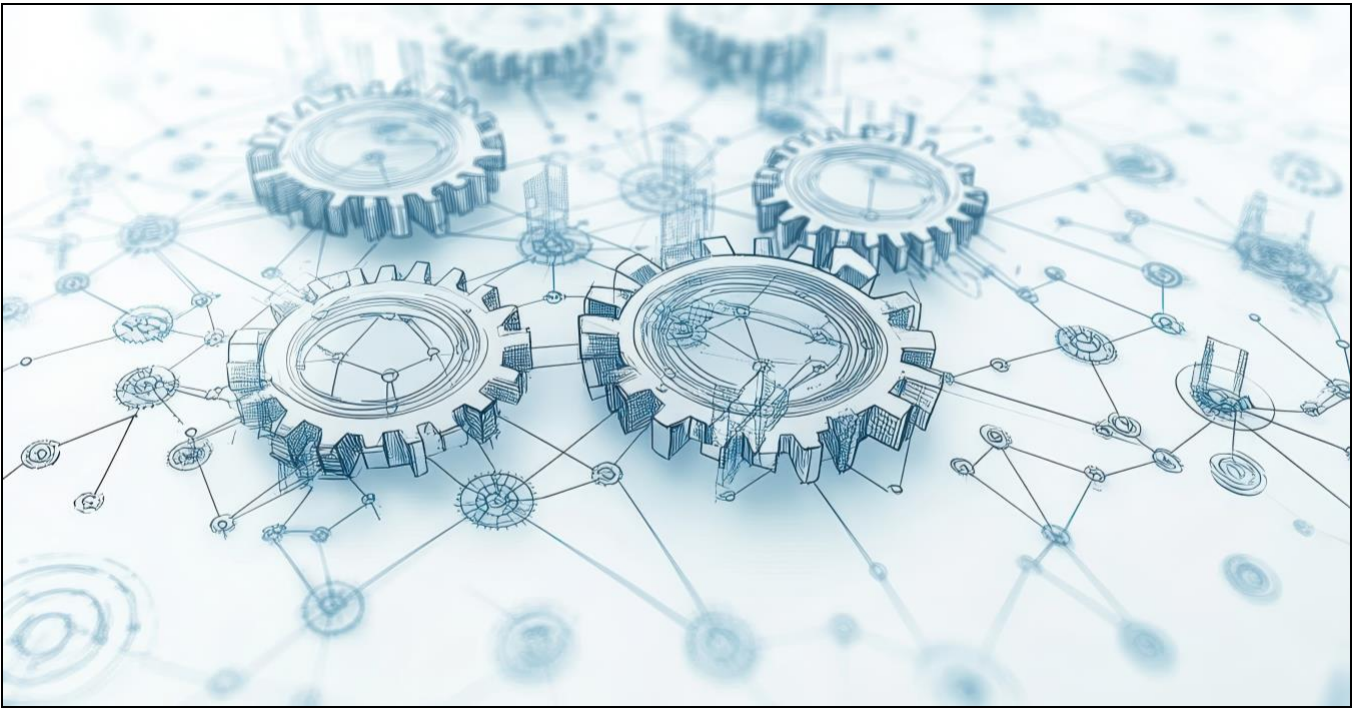


Level 4

Watsonx Orchestrate

Hands-on lab guide: Collaborating agents with Python and Langfuse



Version 3.1

July 2026

LEGAL NOTICES

This information was developed for products and services offered in the USA. IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service. IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing IBM Corporation
North Castle Drive, MD-NC119 Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you. This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice. Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk. IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you. Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All names and references for organizations and other business institutions used in this deliverable's scenarios are fictional. Any match with real organizations or institutions is coincidental. All names and associated information for people in this deliverable's scenarios are fictional. Any match with a real person is coincidental.

TRADEMARKS

IBM, the IBM logo, and [ibm.com](https://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copyright-trademark>.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates. INFOBLOX is a trademark of Infoblox Inc. or its affiliated companies, registered in the United States and other countries.

Microsoft®, Azure, Windows, Windows NT, Windows Server, Microsoft® Excel®, Microsoft® Word®, Microsoft® Outlook®, Microsoft® Excel® logo, Microsoft® Word® logo, Microsoft® Outlook® logo, Microsoft® Windows App, Microsoft® Windows App logo and the Windows logo are trademarks of Microsoft® Corporation in the United States, other countries, or both.

Salesforce and Salesforce logo are registered trademarks of SALESFORCE, INC. (CORPORATION; CALIFORNIA, USA).

Slack is a registered trademark of Salesforce, Inc (CORPORATION; DELAWARE, USA).

Red Hat®, JBoss®, OpenShift®, Fedora®, Hibernate®, Ansible®, CloudForms®, RHCA®, RHCE®, RHCSA®, Ceph®, and Gluster® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Google, Google logo, Google Tasks, Google Cloud Platform, GMAIL, GMAIL logo are registered trademarks of GOOGLE LLC and this material is not endorsed by or affiliated with Google in any way.

Elasticsearch, Elastic search logo, Elastic, Elastic logo are registered trademarks of Elasticsearch BV (LIMITED LIABILITY COMPANY; NETHERLANDS) in United States, other countries or both.

Milvus, Milvus logo are registered trademarks of LF Projects, LLC (LIMITED LIABILITY COMPANY; DELAWARE, USA) in United States, other countries or both.

Meta, Meta logos, Llama, Llama logos are registered trademarks of META PLATFORMS, INC. (CORPORATION; California, USA) in United States, other countries or both.

Mistral, Mistral logo, Mistral AI, Mistral AI logo are registered trademarks of Mistral AI (SOCIÉTÉ PAR ACTIONS SIMPLIFIÉE (SAS); FRANCE) in United States, other countries or both.

PyCharm, PyCharm logo are registered trademarks of JetBrains s.r.o. (CORPORATION; CZECH REPUBLIC) in United States, other countries or both.

Bruno is a trademark held by Anoop M D. Bruno logo is sourced from OpenMoji. License: CC BY-SA 4.0

Apple, Apple logo, Apple store, App store, App store logo are registered trademarks of Apple Inc. (CORPORATION; CALIFORNIA, USA) in United States, other countries or both.

VMWARE, VMWARE logos are registered trademarks of VMware LLC (LIMITED LIABILITY COMPANY; Delaware, USA) in United States, other countries or both.

RANCHER, RANCHER logo, Rancher Desktop, Rancher Desktop logo are registered trademarks of SUSE LLC (LIMITED LIABILITY COMPANY; MASSACHUSETTS, USA) in United States, other countries or both.

Langfuse, Langfuse logo are registered trademarks of Langfuse GmbH (GESELLSCHAFT MIT BESCHRÄNKTER HAFTUNG (GMBH); Germany, Germany) in United States, other countries or both.

OpenMeteo is a registered trademark owned by Patrick Zippenfenig (Individual, Germany) in United States, other countries or both.

© Copyright International Business Machines Corporation 2025. This document may not be reproduced in whole or in part without the prior written permission of IBM.

Table of Contents

- Introduction 1**
 - Lab overview.....1
 - What you will learn2
 - Environment2
 - How to get support.....2
 - Prerequisites2
- Life insurance agent..... 3**
 - Tool and agent creation3
 - Agent testing7
- Monthly payment agent 10**
 - Tool and agent creation 10
 - Agent creation and testing 13
- Collaborating agents 18**
 - Manager agent 18
 - Testing the manager agent 20
- Langfuse trace..... 22**
 - Access the trace 22
 - Analyze the trace 25
- Summary..... 33**
- Appendix: Troubleshooting..... 34**
 - System unresponsive..... 34
 - LLM Error..... 34
- Appendix: Reset your environment 35**

Introduction

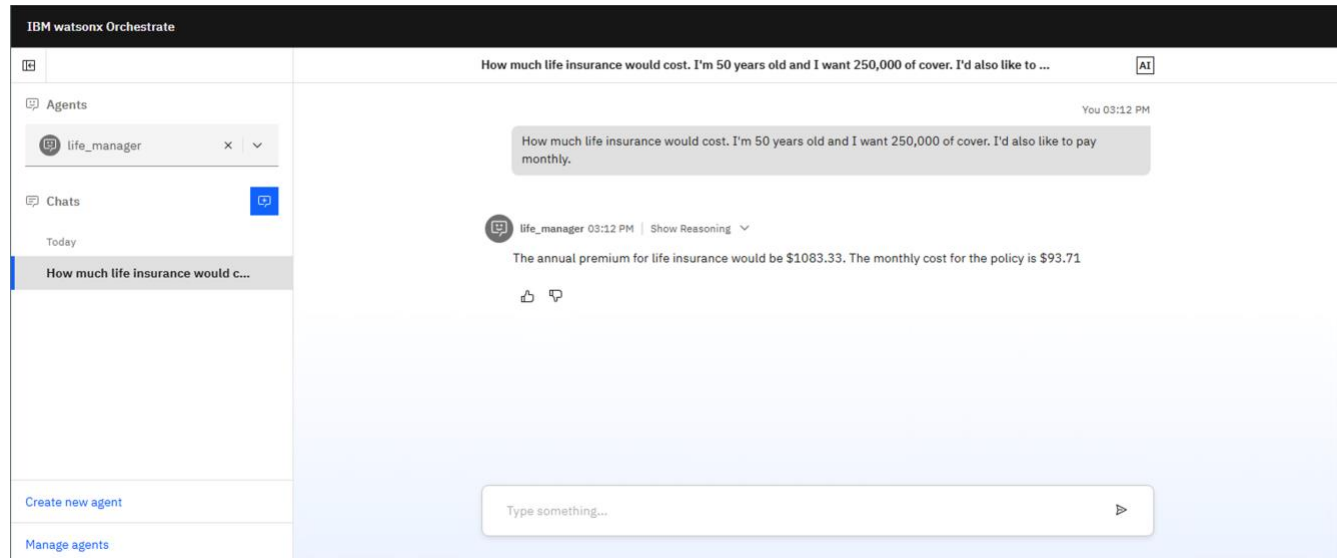
Lab overview

This lab will show you how to create an agentic solution that utilizes collaborating agents and three simple Python tools to provide coverage quotations and payment illustrations for a hypothetical insurance provider.

When complete, the top-level agent will be able to respond to complex requests that contain multiple instructions. The agent will analyze the request and work towards a solution, using its collaborating agents and their tools to produce a response. In the example below, the agent has identified the two parameters needed to calculate the cost of the insurance cover that have been included the users request, these are:

- The amount of cover required
- The customers age

Additionally, the agent has determined that the customer has asked to pay for the product monthly. To respond to this additional requirement, the agent will transfer control to another collaborating agent that has specific knowledge and tools to provide monthly payment illustrations. Once the monthly payment illustration has been calculated, control is passed back to the top-level agent and the response below is generated.



What you will learn

Once you have completed this lab you will have developed skills enabling you to complete the following tasks:

1. Create tools from Python functions and import them into Orchestrate.
2. Create an agent that uses Python based tools.
3. Create a manager agent that routes to collaborating.
4. Review agent description and instructions that define their behavior.
5. Review trace information that explains the agents reasoning and chain of thought.

Environment

This lab has been tested with the Agent Development Kit (ADK) version 2.12 or higher.

How to get support

If you encounter an issue with this lab,

- IBM employees can open an IBM Support ticket at <http://ibm.biz/support>.
- Business Partners can request help at the [Partner Plus Support](#) website.

Prerequisites

Make sure the following prerequisites are satisfied before proceeding with this lab:

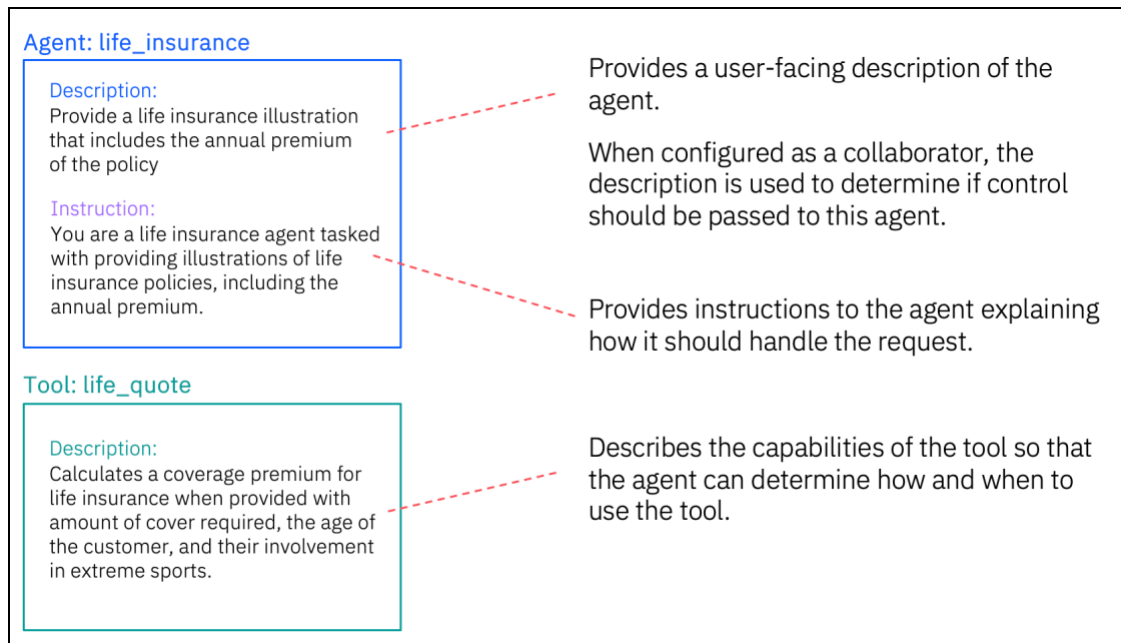
- You have performed the Preparation Lab guide including the environment validation step
- You have your local ADK 2.12 (or higher) environment up and running (Orchestrate server started after validating the environment in the preparation guide OR after having performed another ADK related lab)

Note: If for some reason your server is not started correctly, reset your server following the [Reset your environment](#) appendix.

The tools created in this lab are based on Python functions, but knowledge of Python programming is not required for this lab.

Life insurance agent

In this section you will create and test the first life insurance agent. You will create a single agent and provide this agent with a single tool based on a Python function. The diagram below provides an overview of the configuration used in this section. Review the agent and tool configuration shown below before proceeding.

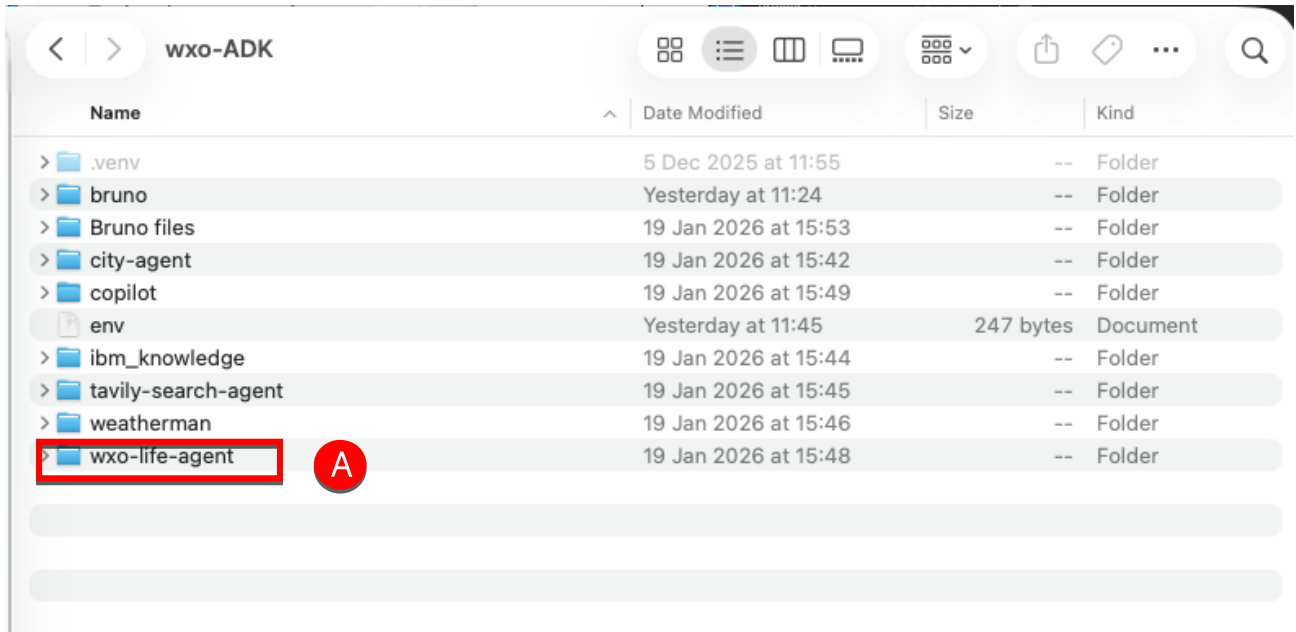


In addition to the tool description, any input and output parameters used by the tool are analyzed and added to the tool configuration. At runtime, the agent shares this configuration as part of the system prompt and is used for planning.

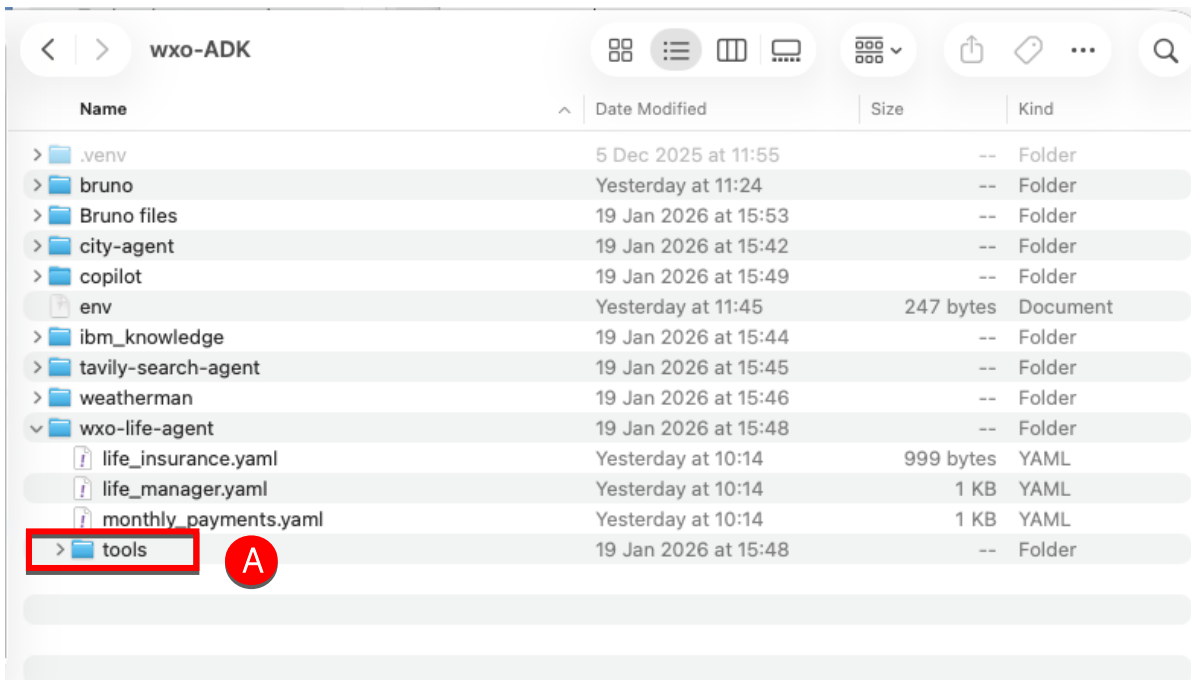
Tool and agent creation

In this section you will review the Python code used by the tool and create the tool and agent in watsonx Orchestrate using the command line interface.

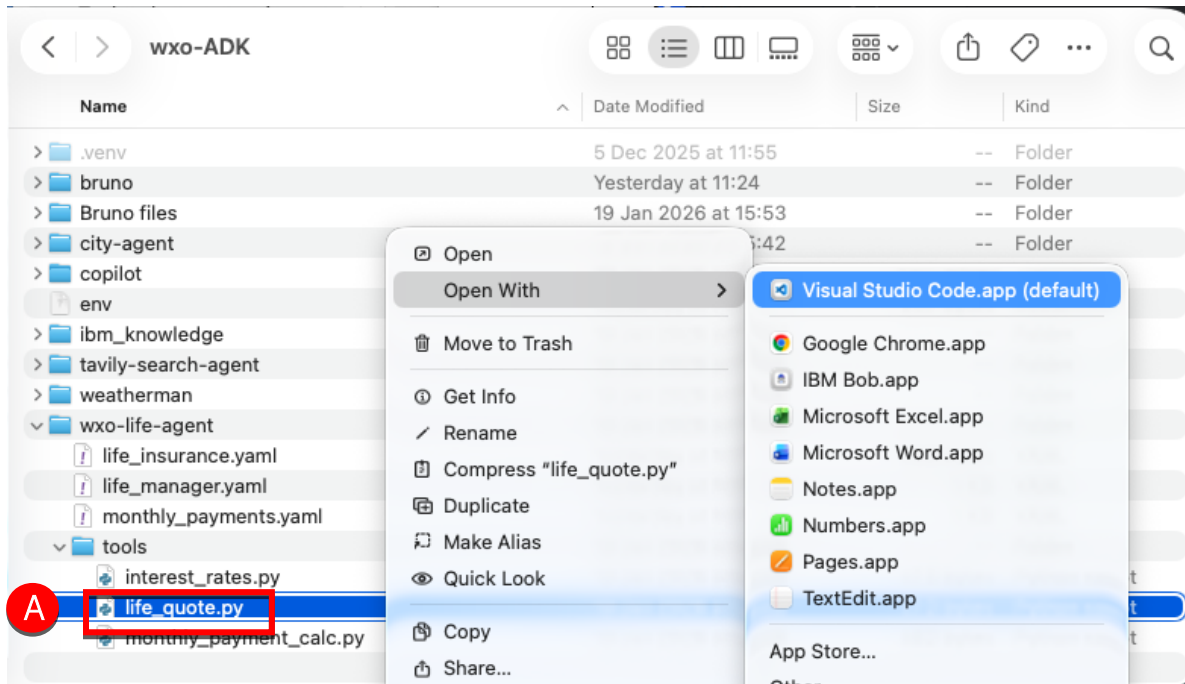
1. Open the ADK environment and start the watsonx Orchestrate server if not yet done. Refer to the [Reset your environment appendix](#) if your orchestrate server is not started.
1. In your file browser, open your **wxo-ADK/wxo-life-agent** folder (A).



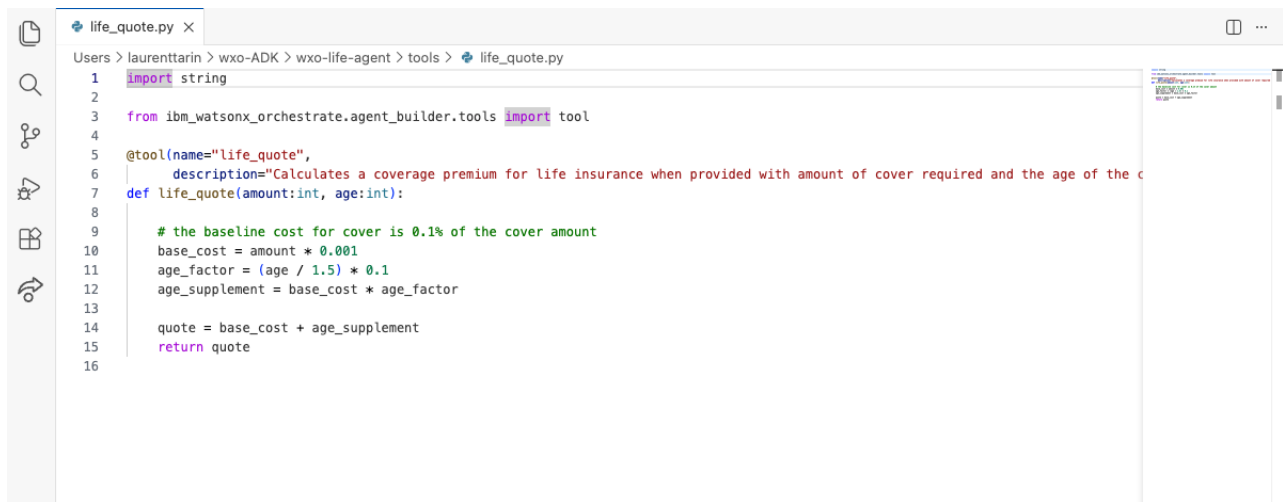
2. Expand the **tools** folder (A).



3. Edit the `life_quote.py` file (A) with your preferred text editor.



4. Review the implementation of the Python tool (this is explained below).



The `@tool` annotation on line 5 identifies this function as an Orchestrate tool, this annotation also provides the **name** of the tool and a **description** of its capabilities. The Python function starts on line 7 and implements a simplistic quotation calculation based on 0.1% of the amount of cover with supplements added for age.

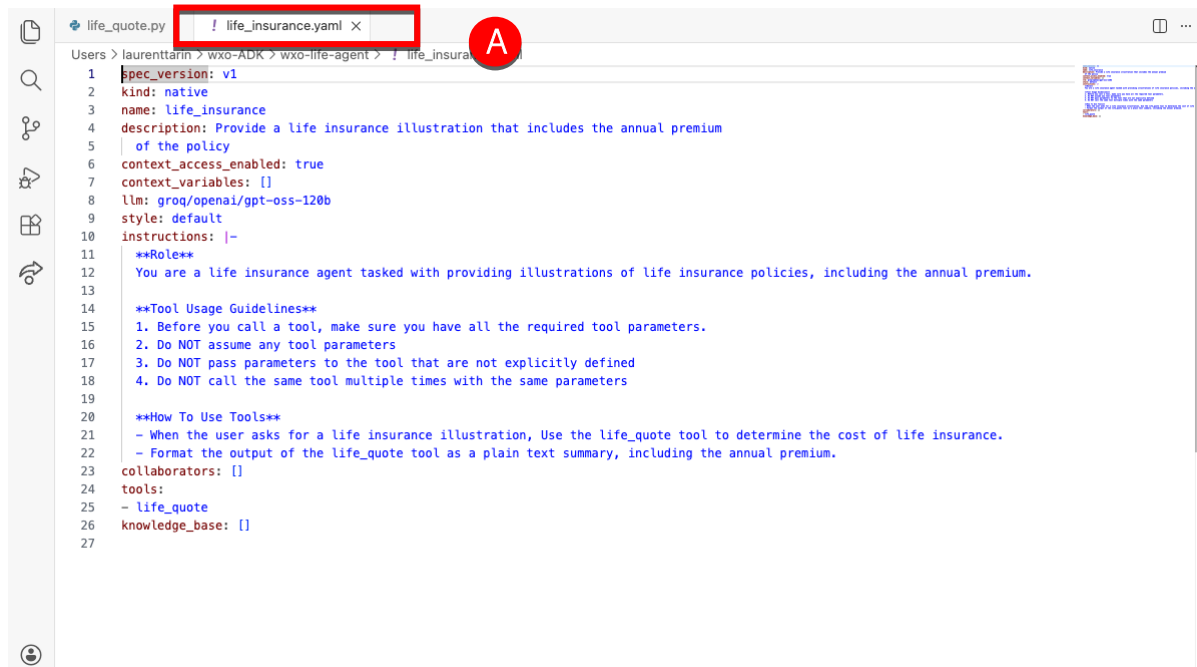
The function receives two parameters, the **amount** of cover required, and the **age** of the customer. When the tool is created, information about these parameters will be added to the configuration and available to any agent using this tool.

5. In your ADK terminal window, enter the following command into the terminal (A) to create the tool.

```
orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
```



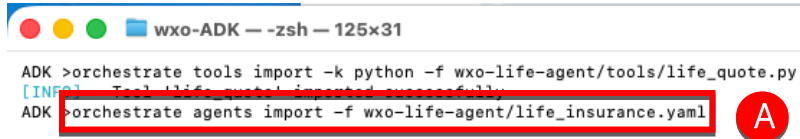
6. Edit the **life_insurance.yaml** (A) and review the definition of the agent.



This yaml file defines the agent, its description behavior/instructions, and the LLM it will use for planning and reasoning. This agent has no collaborator, but a single tool is declared.

7. Enter the following command into the terminal (A) to create the agent.

```
orchestrate agents import -f wxo-life-agent/life_insurance.yaml
```



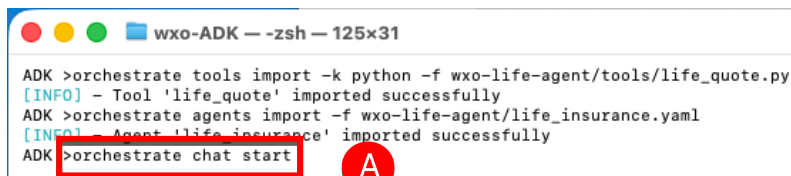
```
wxo-ADK — -zsh — 125x31
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK >orchestrate agents import -f wxo-life-agent/life_insurance.yaml
```

Agent testing

In this section you will start the chat interface, test the agent and review its reasoning in the chat UI.

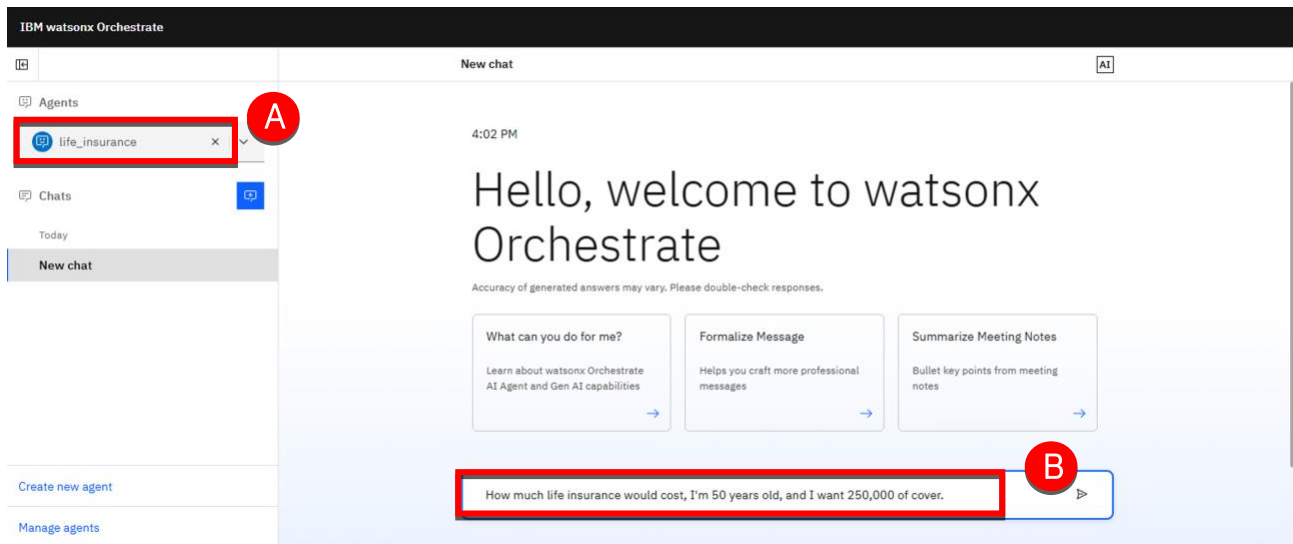
1. Enter the following command into the terminal (A) to start the chat server.

```
orchestrate chat start
```

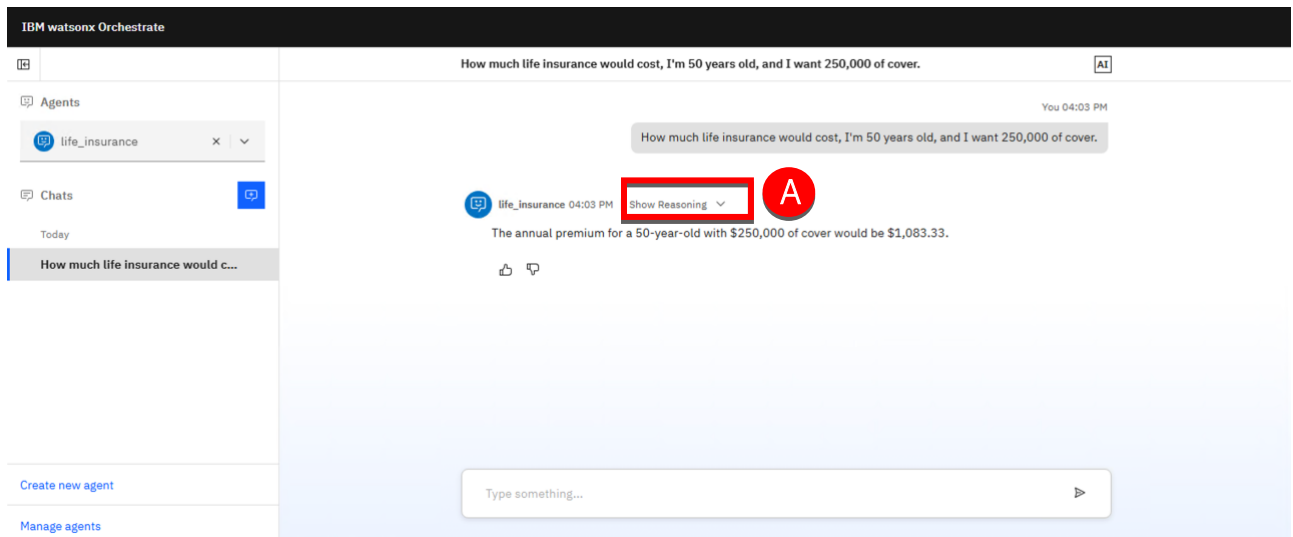


```
wxo-ADK — -zsh — 125x31
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK >orchestrate agents import -f wxo-life-agent/life_insurance.yaml
[INFO] - Agent 'life_insurance' imported successfully
ADK >orchestrate chat start
```

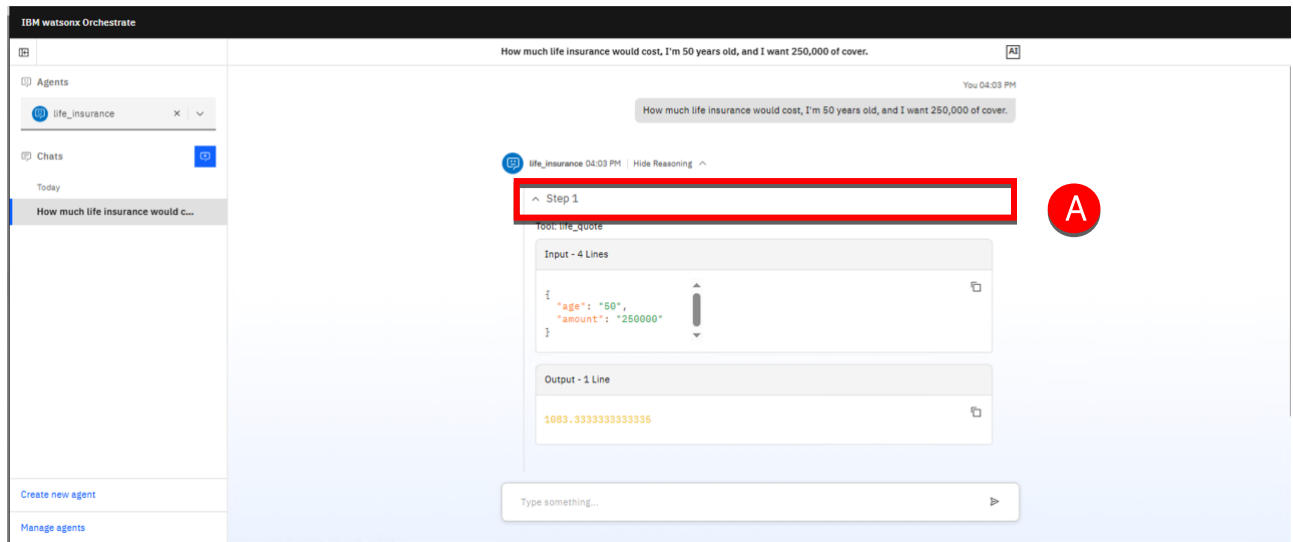
- When the browser opens, ensure **life_insurance** is selected (A) then enter "How much life insurance would cost, I'm 50 years old, and I want 250,000 of cover." into the chat window (B), then press Enter. If the agent is not available, refresh your browser.



The user input is interpreted as a request for a quotation for life insurance. When the agent responds with a quotation, expand the **Show Reasoning** panel (A).



- Expand the **Step 1** panel (A).

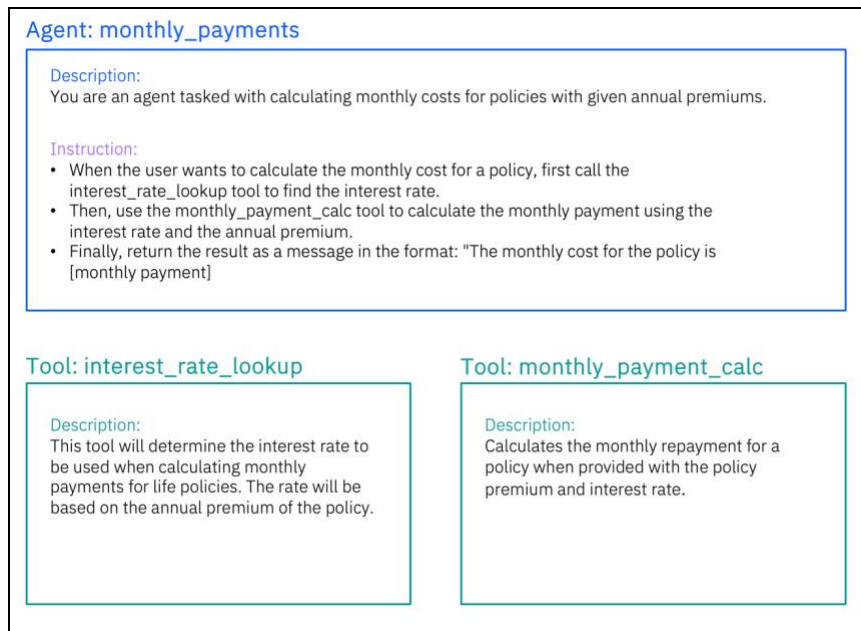


The reasoning panel shows the input and output parameters passed between the agent and the tool.

Monthly payment agent

The solution will now be extended. You will create a second agent that specializes in creating payment illustrations should a customer want to pay for their policies in monthly instalments. This second agent will use two tools, to do this, first, the `interest_rate_lookup` tool will be used to determine the amount of interest to be charged, then the `monthly_payment_calc` tool will be used to calculate the monthly payment.

The diagram below provides an overview of the new agent configuration.



Tool and agent creation

In this section you will review the Python code used by the tools and create them in watsonx Orchestrate using the command line interface.

1. Edit and review the `interest_rates.py` file (A) contained in your `wxo-ADK` folder.

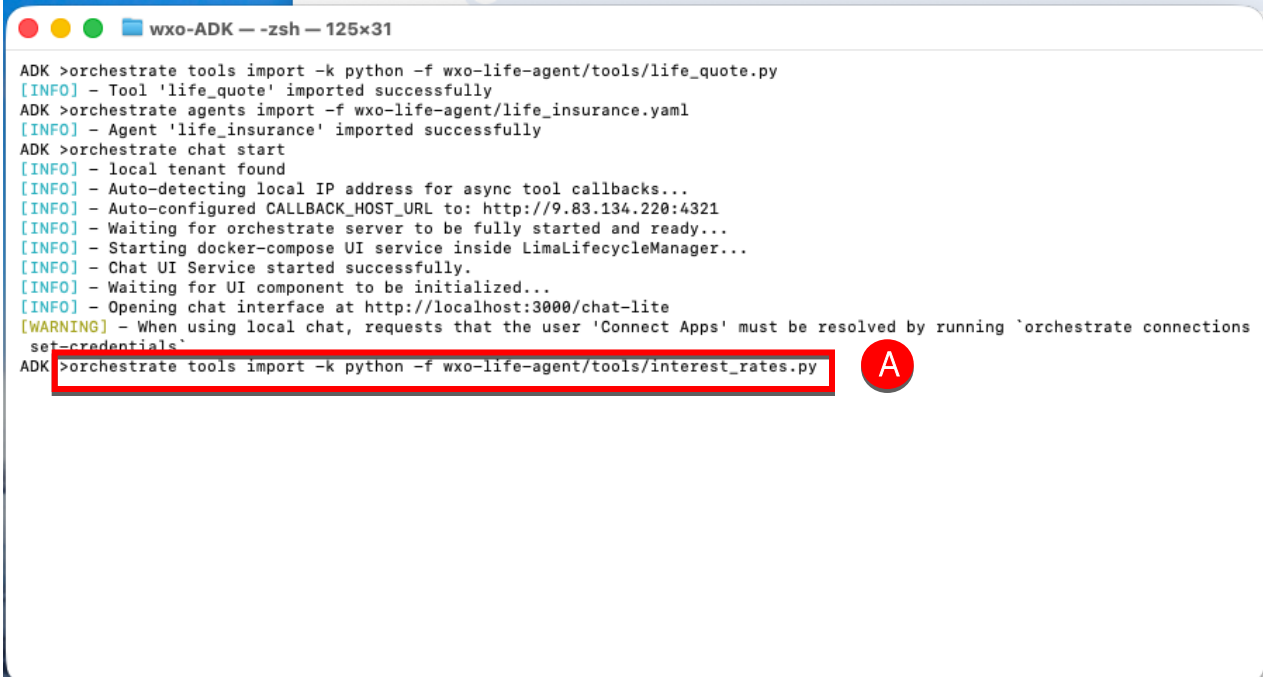
```
1 import random
2 from ibm_watsonx_orchestrate.agent_builder.tools import tool
3
4 @tool(name="interest_rate_lookup",
5       description="This tool will determine the interest rate to be used when calculating monthly payments for policies. The rate will be based on the annual premium of the policy.")
6 def rate_lookup(premium:float):
7     rate = 15
8
9     rate = random.randint(50, 100)
10    rate = rate / 10
11    return rate
12
13
14
```

The **@tool** annotation on line 4 identifies the function on line 6 as an Orchestrate tool. This annotation also provides the **name** of the tool and a **description** of its capabilities. The Python function generates a random interest rate lookup between 5-10%. The policy premium is passed as a parameter into the function, but it is not used.

This function is simply a random number generator, and as such, the calculations performed using its response will vary.

2. Enter the following command into the terminal (A) to import the interest rate lookup tool.

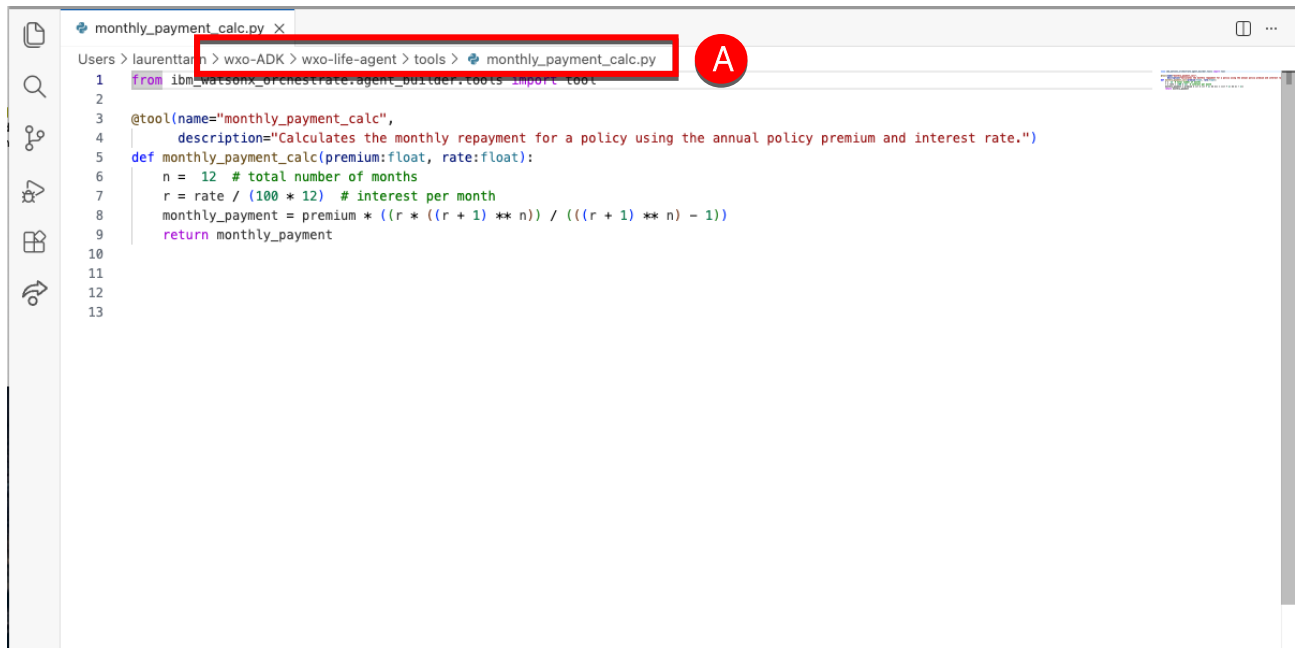
```
orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py
```



A terminal window titled "wxo-ADK -- zsh -- 125x31" displays a series of commands and their outputs. The commands include importing the 'life_quote' tool, importing agents, starting the chat, and starting the chat UI service. The final command, `orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py`, is highlighted with a red box and labeled with a red circle containing the letter 'A'. The terminal output shows various status messages, including successful imports and warnings about local chat usage.

```
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK >orchestrate agents import -f wxo-life-agent/life_insurance.yaml
[INFO] - Agent 'life_insurance' imported successfully
ADK >orchestrate chat start
[INFO] - local tenant found
[INFO] - Auto-detecting local IP address for async tool callbacks...
[INFO] - Auto-configured CALLBACK_HOST_URL to: http://9.83.134.220:4321
[INFO] - Waiting for orchestrate server to be fully started and ready...
[INFO] - Starting docker-compose UI service inside LimaLifecycleManager...
[INFO] - Chat UI Service started successfully.
[INFO] - Waiting for UI component to be initialized...
[INFO] - Opening chat interface at http://localhost:3000/chat-lite
[WARNING] - When using local chat, requests that the user 'Connect Apps' must be resolved by running `orchestrate connections
set-credentials`
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py
```

3. Edit and review the `monthly_payment_calc.py` file contained in your wxo-ADK folder (A).

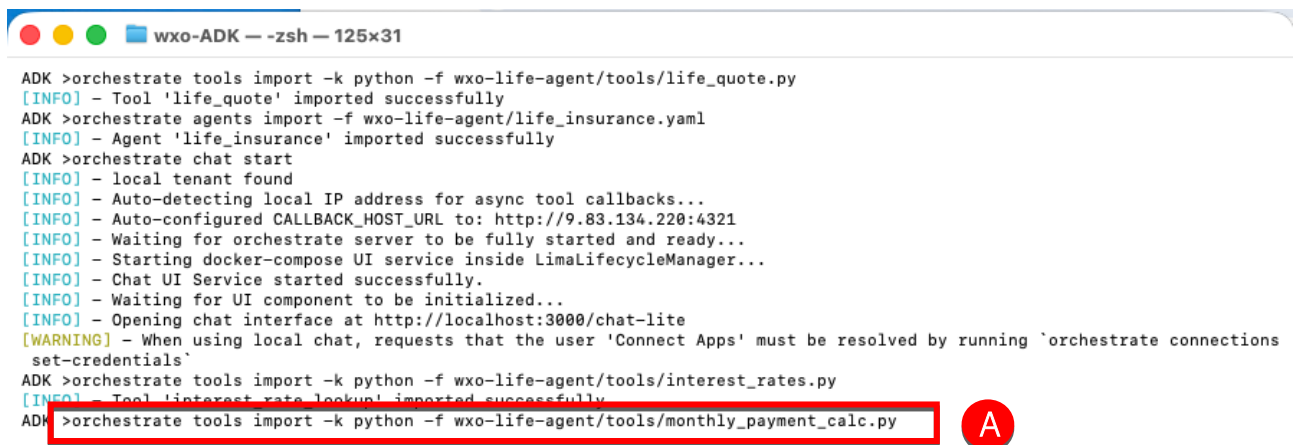


```
1 from ibm_watsonx_orchestrate.agent_builder.tools import tool
2
3 @tool(name="monthly_payment_calc",
4       description="Calculates the monthly repayment for a policy using the annual policy premium and interest rate.")
5 def monthly_payment_calc(premium:float, rate:float):
6     n = 12 # total number of months
7     r = rate / (100 * 12) # interest per month
8     monthly_payment = premium * ((r * ((r + 1) ** n)) / (((r + 1) ** n) - 1))
9     return monthly_payment
10
11
12
13
```

This function calculates a monthly payment based on the premium and interest rate.

4. Enter the following command into the terminal (A) to import the interest rate lookup tool.

```
orchestrate tools import -k python -f wxo-life-agent/tools/monthly_payment_calc.py
```

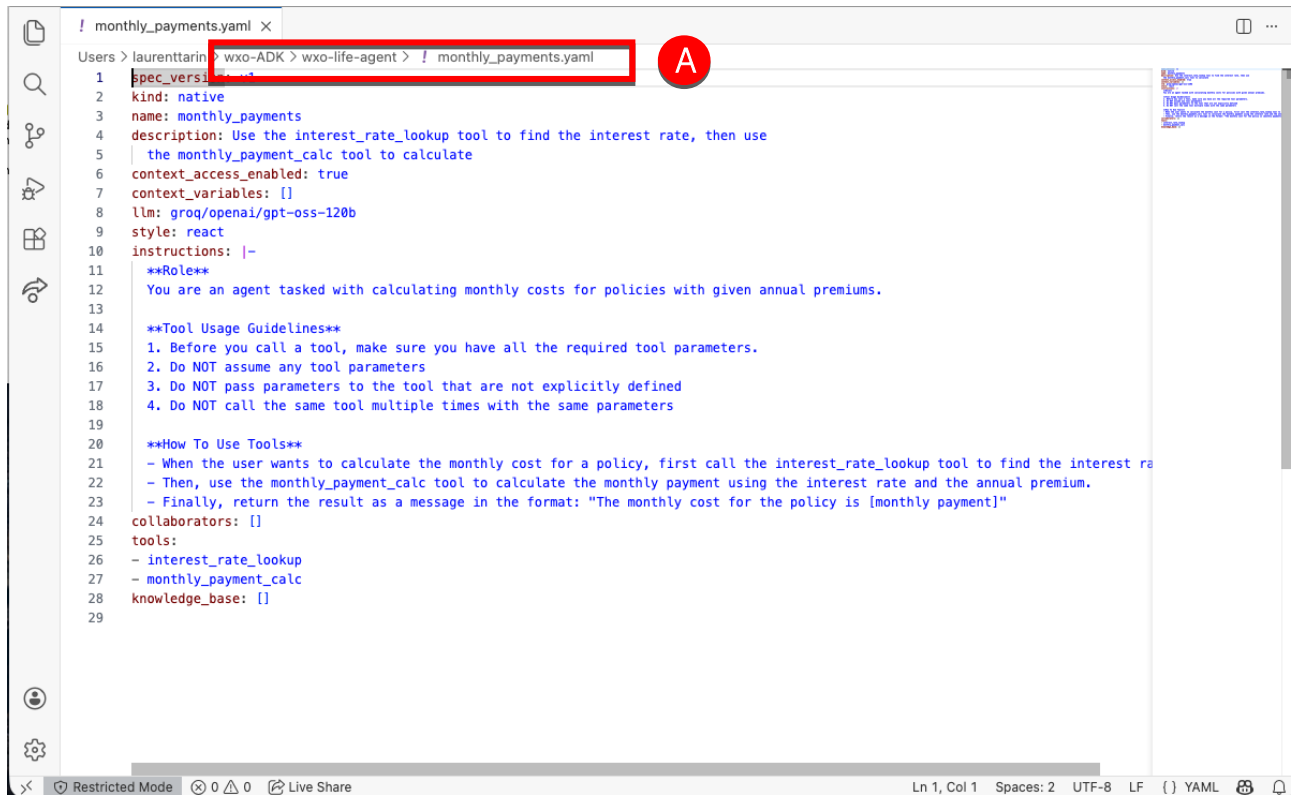


```
wxo-ADK --zsh 125x31
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK > orchestrate agents import -f wxo-life-agent/life_insurance.yaml
[INFO] - Agent 'life_insurance' imported successfully
ADK > orchestrate chat start
[INFO] - local tenant found
[INFO] - Auto-detecting local IP address for async tool callbacks...
[INFO] - Auto-configured CALLBACK_HOST_URL to: http://9.83.134.220:4321
[INFO] - Waiting for orchestrate server to be fully started and ready...
[INFO] - Starting docker-compose UI service inside LimaLifecycleManager...
[INFO] - Chat UI Service started successfully.
[INFO] - Waiting for UI component to be initialized...
[INFO] - Opening chat interface at http://localhost:3000/chat-lite
[WARNING] - When using local chat, requests that the user 'Connect Apps' must be resolved by running `orchestrate connections set-credentials`
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py
[INFO] - Tool 'interest_rate_lookup' imported successfully
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/monthly_payment_calc.py
```

Agent creation and testing

In this section you will review the definition of the agent and create it in watsonx Orchestrate using the command line interface. Once the agent has been created, you will test the agent and review its reasoning.

5. Edit and review the **monthly_payments.yaml** (A) file contained in your wxo-ADK folder.



```
1 spec_version: 1.0
2 kind: native
3 name: monthly_payments
4 description: Use the interest_rate_lookup tool to find the interest rate, then use
5   the monthly_payment_calc tool to calculate
6 context_access_enabled: true
7 context_variables: []
8 llm: groq/openai/gpt-oss-120b
9 style: react
10 instructions: |-
11   **Role**
12   You are an agent tasked with calculating monthly costs for policies with given annual premiums.
13
14   **Tool Usage Guidelines**
15   1. Before you call a tool, make sure you have all the required tool parameters.
16   2. Do NOT assume any tool parameters
17   3. Do NOT pass parameters to the tool that are not explicitly defined
18   4. Do NOT call the same tool multiple times with the same parameters
19
20   **How To Use Tools**
21   - When the user wants to calculate the monthly cost for a policy, first call the interest_rate_lookup tool to find the interest rate
22   - Then, use the monthly_payment_calc tool to calculate the monthly payment using the interest rate and the annual premium.
23   - Finally, return the result as a message in the format: "The monthly cost for the policy is [monthly payment]"
24 collaborators: []
25 tools:
26   - interest_rate_lookup
27   - monthly_payment_calc
28 knowledge_base: []
29
```

This yaml file defines the agent, its description and behavior, and the LLM it uses for planning and reasoning.



Most LLM's will try and generate an approximate monthly payment. This approximate calculation may appear reasonable, but it won't be based on an accurate amortization formula like the one used in the tool. As LLM's become more capable, identifying these hallucinations is increasingly difficult.

This agent includes additional instructions on how to use its tools correctly and format the response.

```

instructions: |-
  **Role**
  You are an agent tasked with calculating monthly costs for policies with given annual premiums.

  **Tool Usage Guidelines**
  1. Before you call a tool, make sure you have all the required tool parameters.
  2. Do NOT assume any tool parameters
  3. Do NOT pass parameters to the tool that are not explicitly defined
  4. Do NOT call the same tool multiple times with the same parameters

  **How To Use Tools**
  - When the user wants to calculate the monthly cost for a policy, first call the interest_rate_lookup tool to find the interest rate.
  - Then, use the monthly_payment_calc tool to calculate the monthly payment using the interest rate and the annual premium.
  - Finally, return the result as a message in the format: "The monthly cost for the policy is [monthly payment]"

```

6. Enter the following command into the terminal (A) to create the agent.

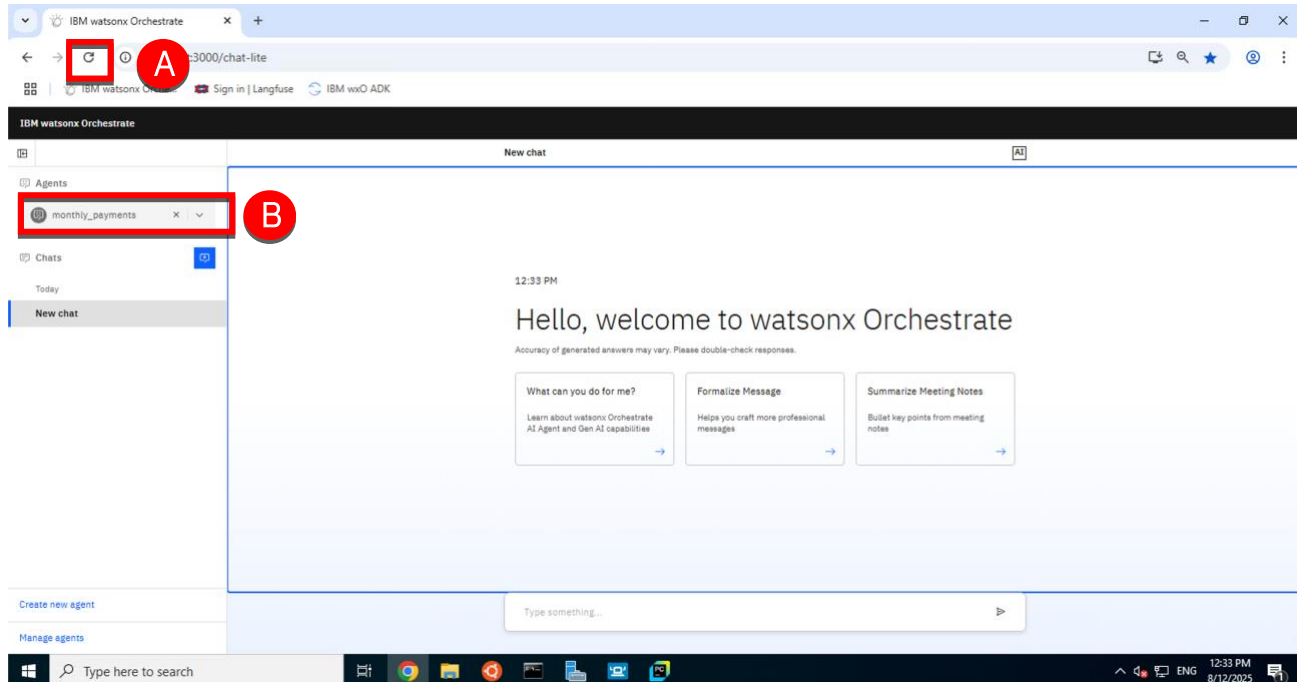
```
orchestrate agents import -f wxo-life-agent/monthly_payments.yaml
```

```

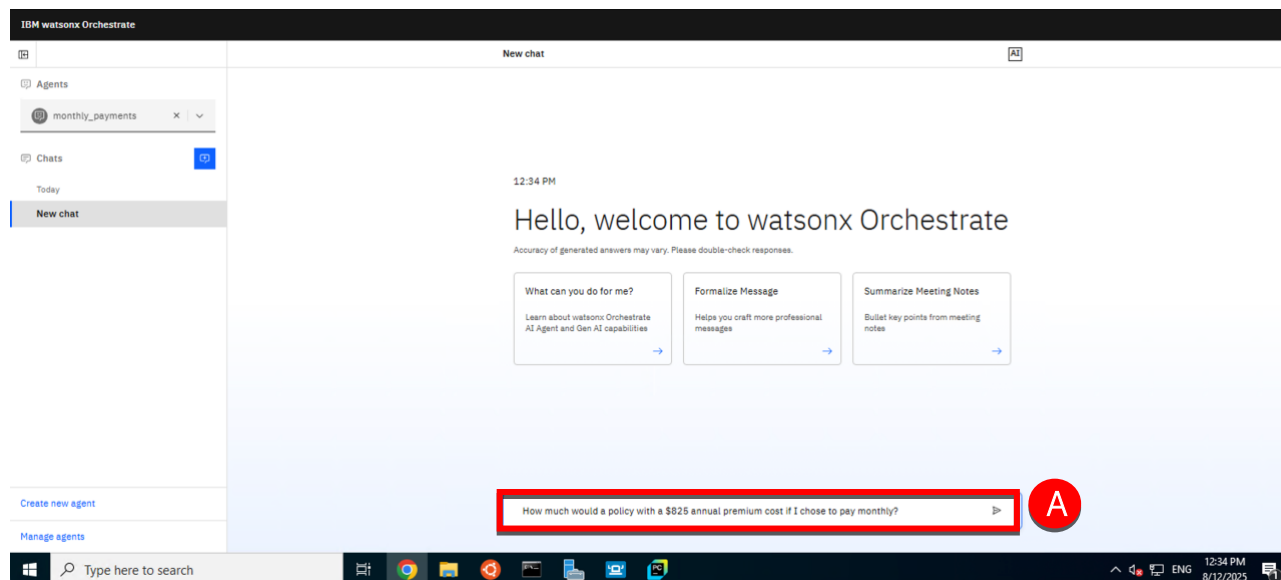
wxo-ADK --zsh -- 125x31
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK >orchestrate agents import -f wxo-life-agent/life_insurance.yaml
[INFO] - Agent 'life_insurance' imported successfully
ADK >orchestrate chat start
[INFO] - local tenant found
[INFO] - Auto-detecting local IP address for async tool callbacks...
[INFO] - Auto-configured CALLBACK_HOST_URL to: http://9.83.134.220:4321
[INFO] - Waiting for orchestrate server to be fully started and ready...
[INFO] - Starting docker-compose UI service inside LimaLifecycleManager...
[INFO] - Chat UI Service started successfully.
[INFO] - Waiting for UI component to be initialized...
[INFO] - Opening chat interface at http://localhost:3000/chat-lite
[WARNING] - When using local chat, requests that the user 'Connect Apps' must be resolved by running `orchestrate connections set-credentials`
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py
[INFO] - Tool 'interest_rate_lookup' imported successfully
ADK >orchestrate tools import -k python -f wxo-life-agent/tools/monthly_payment_calc.py
[INFO] - Tool 'monthly_payment_calc' imported successfully
ADK >orchestrate agents import -f wxo-life-agent/monthly_payments.yaml

```

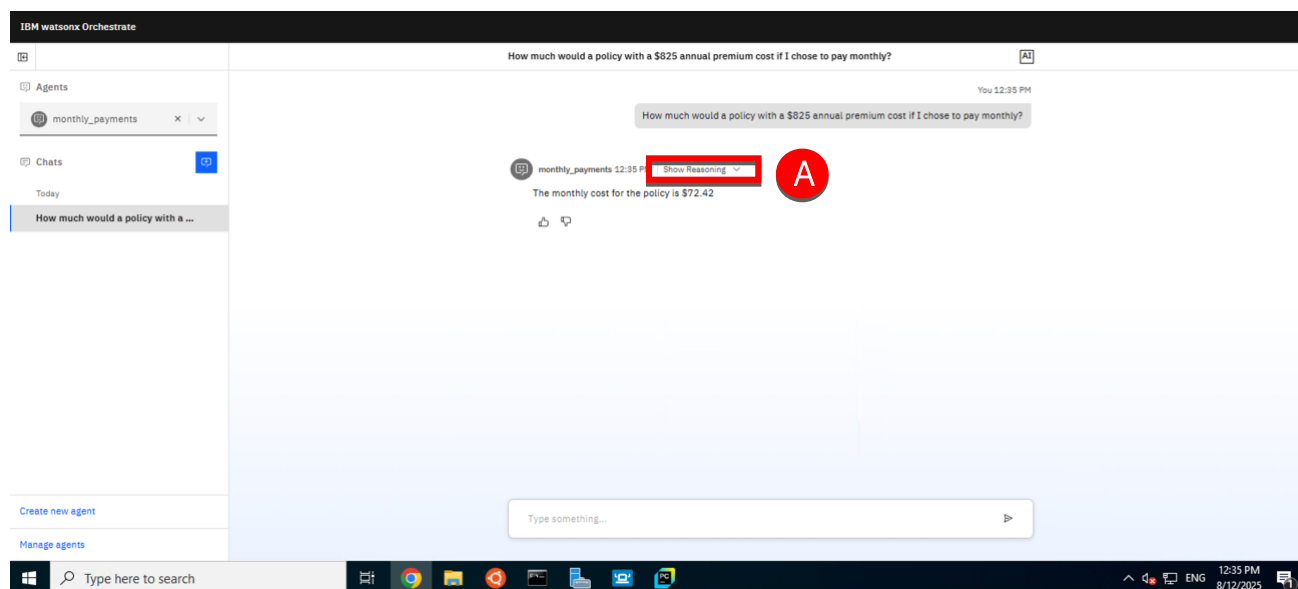
7. In the browser window click the refresh icon (A) then select **monthly_payments** from the **Agents** list (B).



8. Enter "How much would a policy with a \$825 annual premium cost if I chose to pay monthly?" into the chat window (A), then press Enter.



9. When the agent responds expand the **Show Reasoning** panel (A).



10. Expand Step 1 (A).

The screenshot shows the IBM watsonx Orchestrate interface. On the left, there's a sidebar with 'Agents' and 'Chats'. The 'Agents' section shows 'monthly_payments' selected. The 'Chats' section shows a chat titled 'How much would a policy with a ...'. The main area displays a chat conversation. The user's message is 'How much would a policy with a \$825 annual premium cost if I chose to pay monthly?'. The agent's response is 'Step 1'. Below this, the tool used is 'interest_rate_lookup'. The input is a JSON object:

```
{  "premium": "825"}
```

. The output is a single line: 'Observation: 9.7'. A red box highlights the 'Step 1' header, and a red circle with the letter 'A' is next to it.

The agent has begun by using the `interest_rate_lookup` tool (this is a random number).

11. Expand Step 2 (A).

The screenshot shows the IBM watsonx Orchestrate interface. On the left, there's a sidebar with 'Agents' and 'Chats'. The 'Agents' section shows 'monthly_payments' selected. The 'Chats' section shows a chat titled 'How much would a policy with a ...'. The main area displays a chat conversation. The user's message is 'How much would a policy with a \$825 annual premium cost if I chose to pay monthly?'. The agent's response is 'Step 2'. Below this, the tool used is 'monthly_payment_calc'. The input is a JSON object:

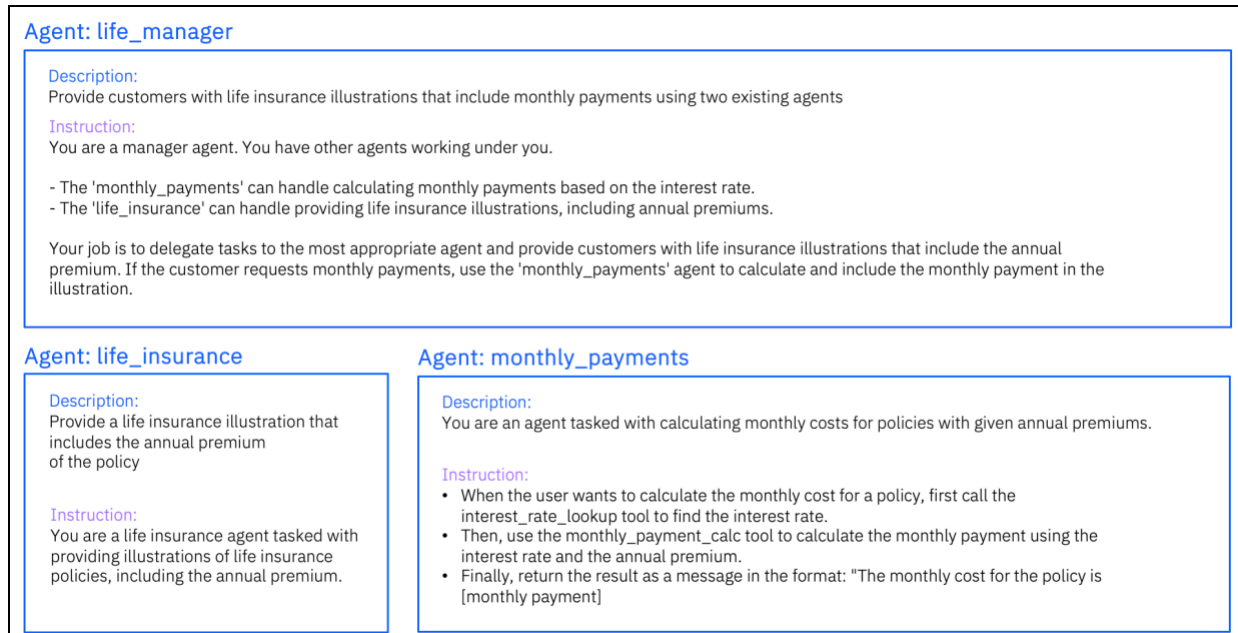
```
{  "rate": "9.7",  "premium": "825"}
```

. The output is a single line: 'Observation: 72.41654742052328'. A red box highlights the 'Step 2' header, and a red circle with the letter 'A' is next to it.

The agent has used the `monthly_payment_calc` tool to determine the monthly payment based on the premium and interest rate and formatted the response according to the instructions.

Collaborating agents

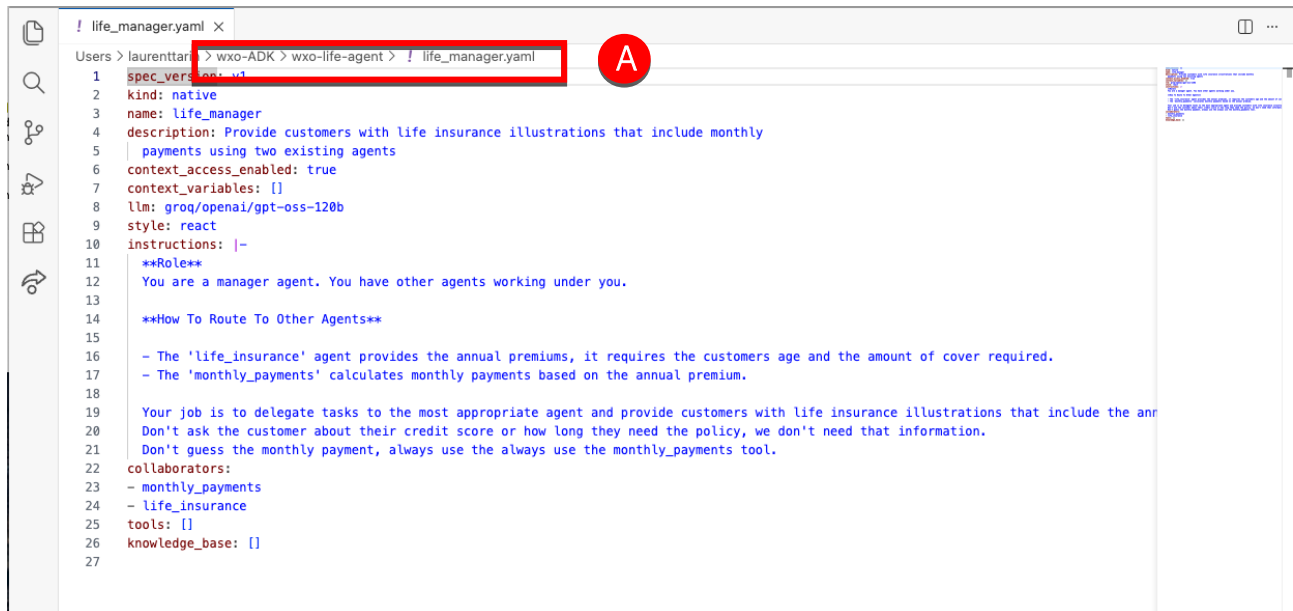
Next you will add a further agent that is able to collaborate with the two existing agents to answer more complex enquires that involve a life cover quotation, combined with monthly payments. The diagram below provides an overview of the expanded configuration used in this section. Tools used by the first two agents have been omitted.



Manager agent

In this section you will review the new agent definition and import it into watsonx Orchestrate using the command line interface.

1. Edit and review the **life_manager.yaml** file (A) contained in your wxo-ADK folder.

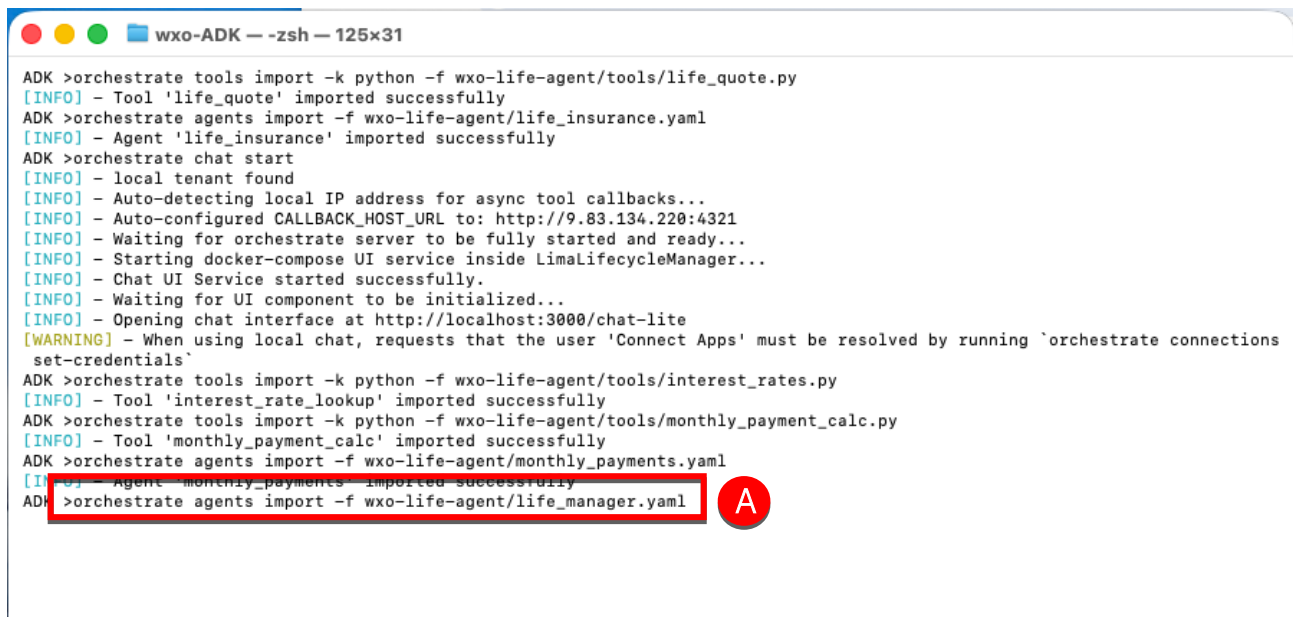


```
! life_manager.yaml x
Users > laurenttar > wxo-ADK > wxo-life-agent > ! life_manager.yaml
1 spec_version: v1
2 kind: native
3 name: life_manager
4 description: Provide customers with life insurance illustrations that include monthly
5   payments using two existing agents
6 context_access_enabled: true
7 context_variables: []
8 llm: groq/openai/gpt-oss-120b
9 style: react
10 instructions: |-
11   **Role**
12   You are a manager agent. You have other agents working under you.
13
14   **How To Route To Other Agents**
15
16   - The 'life_insurance' agent provides the annual premiums, it requires the customers age and the amount of cover required.
17   - The 'monthly_payments' calculates monthly payments based on the annual premium.
18
19   Your job is to delegate tasks to the most appropriate agent and provide customers with life insurance illustrations that include the ann
20   Don't ask the customer about their credit score or how long they need the policy, we don't need that information.
21   Don't guess the monthly payment, always use the always use the monthly_payments tool.
22 collaborators:
23   - monthly_payments
24   - life_insurance
25 tools: []
26 knowledge_base: []
27
```

Review the agent definition and note the addition of a collaborator agents.

2. Enter the following command into the terminal (A) to add the agent.

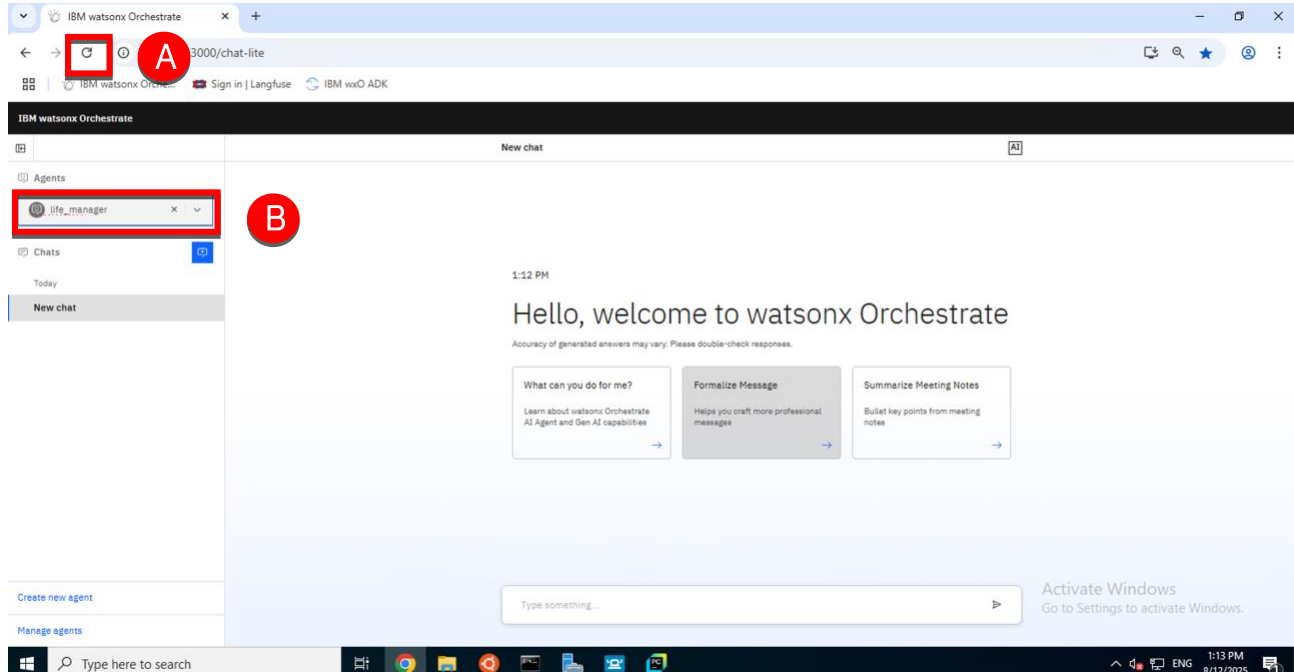
```
orchestrate agents import -f wxo-life-agent/life_manager.yaml
```



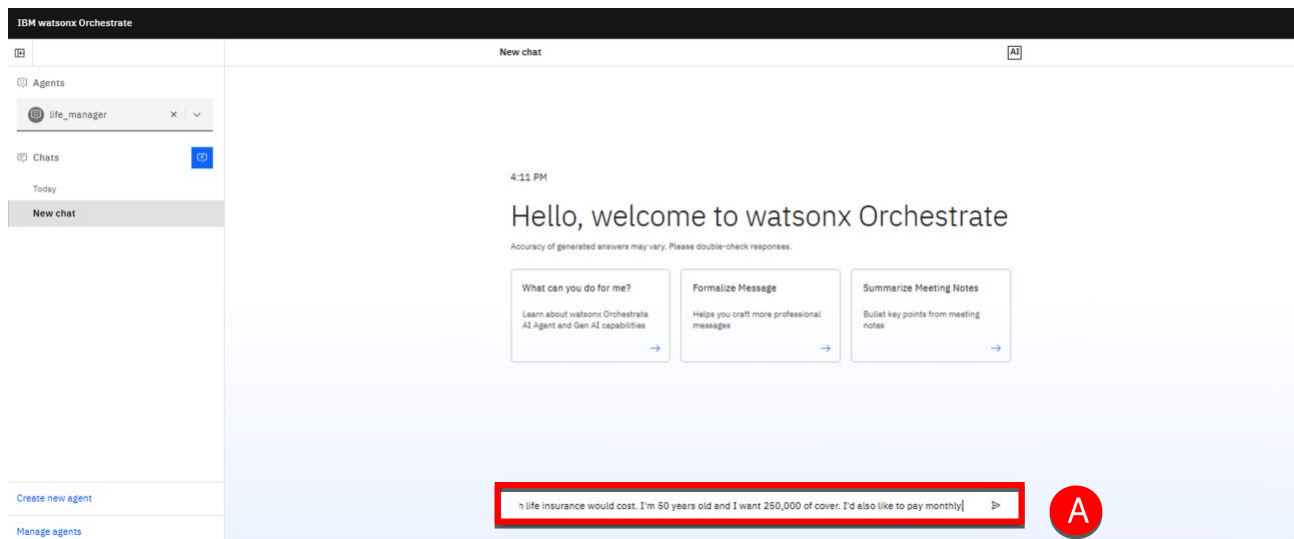
```
wxo-ADK -- zsh -- 125x31
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/life_quote.py
[INFO] - Tool 'life_quote' imported successfully
ADK > orchestrate agents import -f wxo-life-agent/life_insurance.yaml
[INFO] - Agent 'life_insurance' imported successfully
ADK > orchestrate chat start
[INFO] - local tenant found
[INFO] - Auto-detecting local IP address for async tool callbacks...
[INFO] - Auto-configured CALLBACK_HOST_URL to: http://9.83.134.220:4321
[INFO] - Waiting for orchestrate server to be fully started and ready...
[INFO] - Starting docker-compose UI service inside LimaLifecycleManager...
[INFO] - Chat UI Service started successfully.
[INFO] - Waiting for UI component to be initialized...
[INFO] - Opening chat interface at http://localhost:3000/chat-lite
[WARNING] - When using local chat, requests that the user 'Connect Apps' must be resolved by running `orchestrate connections
set-credentials`
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/interest_rates.py
[INFO] - Tool 'interest_rate_lookup' imported successfully
ADK > orchestrate tools import -k python -f wxo-life-agent/tools/monthly_payment_calc.py
[INFO] - Tool 'monthly_payment_calc' imported successfully
ADK > orchestrate agents import -f wxo-life-agent/monthly_payments.yaml
[INFO] - Agent 'monthly_payments' imported successfully
ADK > orchestrate agents import -f wxo-life-agent/life_manager.yaml
```

Testing the manager agent

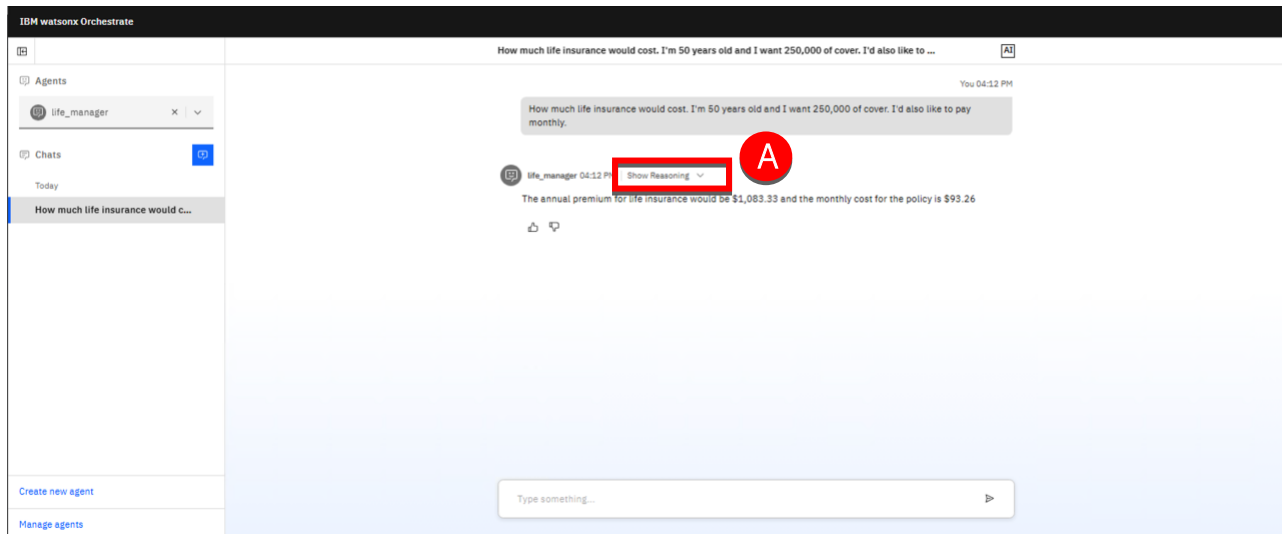
1. Return to the browser window and click refresh (A) and ensure the **life_manager** is selected from the **Agents** list (B).



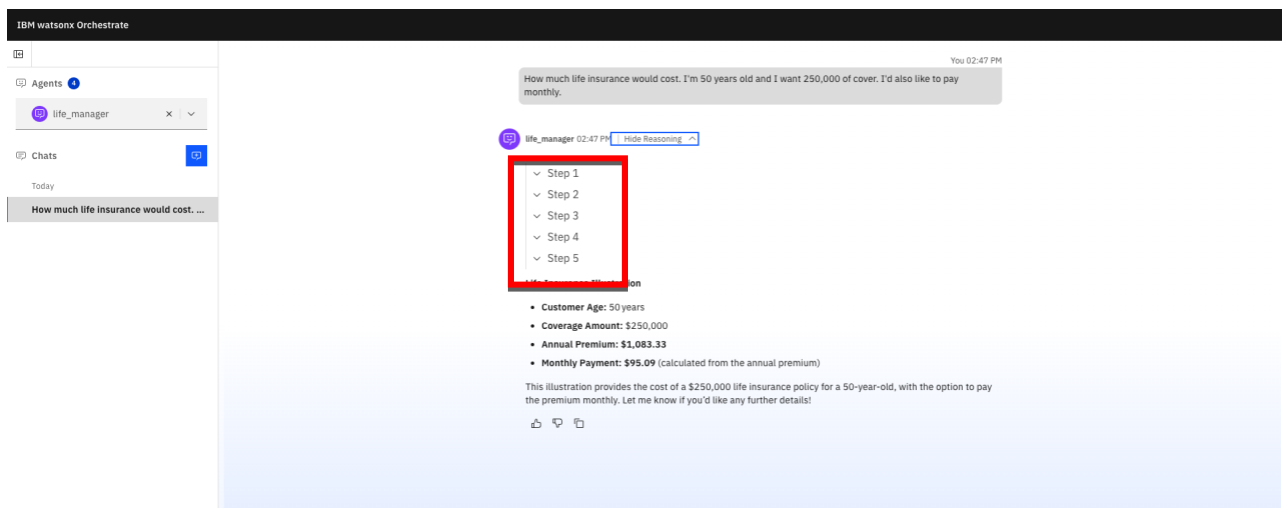
2. Enter "How much life insurance would cost. I'm 50 years old and I want 250,000 of cover. I'd also like to pay monthly." into the chat window (A), then press Enter.



3. When the agent responds, expand the **Show Reasoning** panel (A).



The reasoning for this request has required several steps that are summarized below:



- Step 1: Transfer to agent: life_insurance
- Step 2: Tool call: life_quote
- Step 3: Transfer to agent: monthly_payments
- Step 4: Tool call: interest_rate_lookup
- Step 5: Tool call: monthly_payment_calc

Expand each step to review the tool usage and agent transfer.

It would also have been possible to achieve the same result by adding the additional tools and instructions into a single top-level agent. However, decomposing this functionality into separate agents keeps the agents smaller, simpler, promotes reuse and should improve the reasoning accuracy of the solution.

Langfuse trace

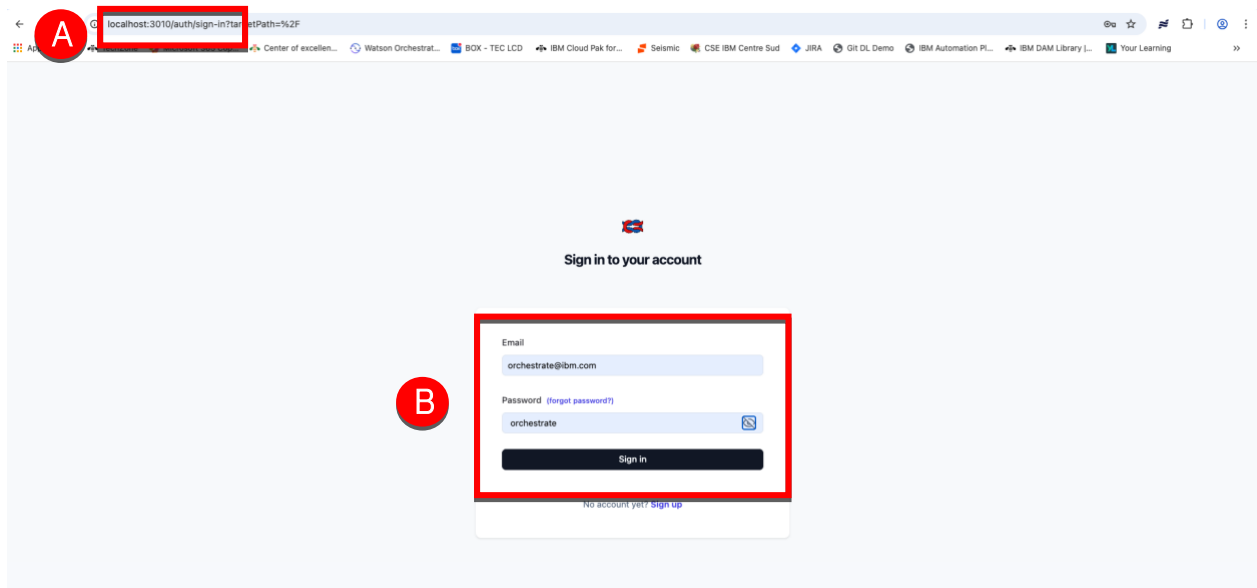
In this section you will review the trace information captured in Langfuse and understand the chain of thought used by the agent to produce the response.

Access the trace

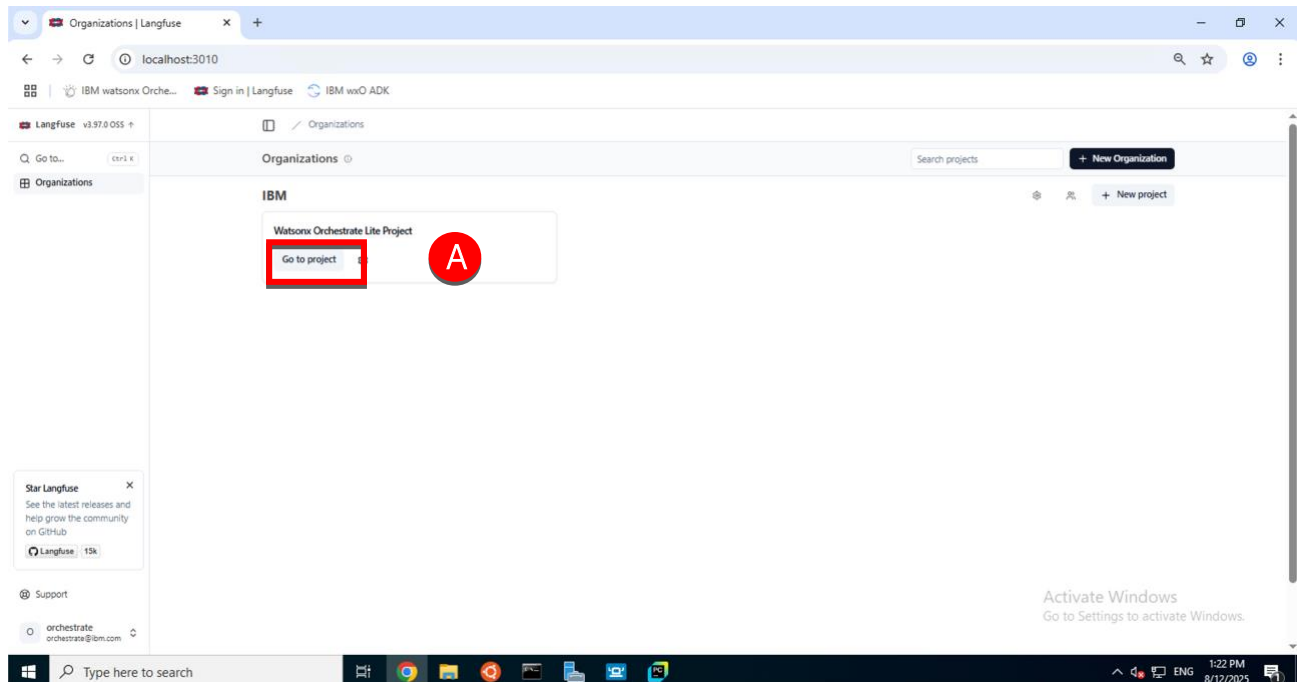
Note: The watsonx orchestrate server has been started with the -l option. This means it was enabling the embedded Langfuse to be started. Links and credentials to login Langfuse are given after the server has started:

```
[INFO] - Waiting for Orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run "orchestrate env activate local" to set your environment of "orchestrate" that start to store the OL service and begin setting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >
```

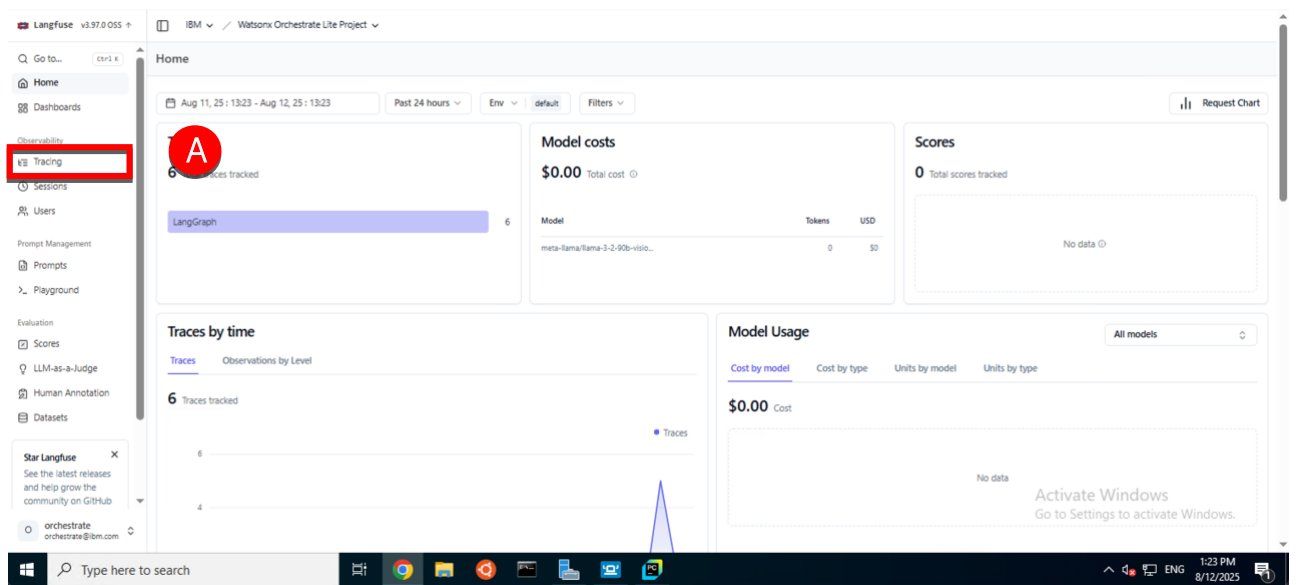
1. Click <http://localhost:3010> (A) and sing in Langfuse using username: orchestrate@ibm.com, password: orchestrate (B).



2. Click **Go to project** (A).



3. Click **Tracing** (A).



4. Click **Hide filters** (A) to hide the side-menu.

The screenshot shows the Langfuse Tracing interface. In the left sidebar, under the 'Tracing' section, there is a button labeled 'Hide filters' (A). The main table displays a list of traces with columns for Timestamp, Name, Input, Output, Observation Levels, Latency, and Tokens. The first trace has a timestamp of 2026-01-21 14:47:40 and a name of LangGraph.

5. Click the 3rd Timestamp of the top trace in the table (A), this will open the trace from the last request. Check that the input (B) corresponds to your last interaction with the chat interface.

The screenshot shows the Langfuse Tracing interface with the 'Hide filters' button (A) in the left sidebar. The main table displays a list of traces. The third trace in the table has a timestamp of 2026-01-21 14:47:34 (A) and an input of 'How much life insurance would cost, I'm 50 years old and I want 25...' (B). The output for this trace is 'The monthly cost for the policy is \$71.42'.

- Click the expand icon (A), this will show the trace in full screen.

The screenshot shows the Langfuse interface with a trace expanded in full screen. The trace details include a graph of the process flow and a detailed view of the 'Monthly_payment_calc' step, showing its input, output, and additional input fields.

Analyze the trace

- Review the Langfuse trace display containing the trace (A), the agent graph (B) and the trace detail (C).

The screenshot shows the Langfuse interface with a trace expanded in full screen. The trace details include a graph of the process flow and a detailed view of the 'Monthly_payment_calc' step, showing its input, output, and additional input fields. Red boxes and letters A, B, and C highlight specific areas: A points to the trace list, B points to the agent graph, and C points to the trace details.

2. Note the Session ID (A).

The screenshot displays the Langfuse interface for a project named 'Watsonx Orchestrate Life Project'. The trace is for a LangGraph application with ID '182fa258-37a0-49d4-8412-32850e4e1092'. The session ID is '6903233f-1b92-43a8-9f46-1ba4e989d65', which is highlighted with a red box and labeled 'A'. The trace shows a sequence of steps: 'start', 'agent (3/3)', 'tools (2/2)', 'answer', and 'end'. The 'agent (3/3)' step is currently selected, showing a detailed view of the agent's output, including a 'Life Insurance Illustration' and 'Additional Input'.

Life Insurance Illustration

- Age: 50
- Coverage Amount: \$250,000
- Annual Premium: \$1,083.33
- Monthly Payment: \$92.94 (calculated from the annual premium)

If you have any further questions or would like to proceed, just let me know!

Additional Input

Path	Value
current_agent	"life_manager"
current_agent_id	"dafac486-bf32-42f7-9ce9-06118abc67a8"
agent_display_name	"life_manager"
reflection_retry_count	0

Multiple steps in the same conversation will have the same session ID. The traces are grouped by Langfuse and can be viewed together by clicking on the session ID. This makes it much easier to debug multi-step conversations. Note that if you start a new chat in Orchestrate, a new session ID is created.

3. Click the first entry called **Invoke_agent** (A). The trace detail displays the **System** prompt used by the agent (B). Click the formatted view icon (C)

The screenshot shows the Langfuse interface for a trace titled 'LangGraph: d67e55fd642bf42ae9c00b8a64c1b9'. The trace details for 'invoke_agent' are displayed, including a graph of the workflow, a list of tools, and a detailed system prompt. The system prompt is highlighted with a red box and labeled 'B'. The user input and assistant output are shown at the bottom, with a formatted view icon labeled 'C'.

System Prompt (B):

Role
You are a manager agent. You have other agents working under you.

How To Route To Other Agents

- The 'life_insurance' agent provides the annual premiums, it requires the customers age and the amount of cover required.
- The 'monthly_payments' calculates monthly payments based on the annual premium.

Your job is to delegate tasks to the most appropriate agent and provide customers with life insurance illustrations that include the annual premium. If the customer requests monthly payments, use the 'interest_rate_monthly_payment' agent to calculate and include the monthly payment in the illustration. Don't ask the customer about their credit score or how long they need the policy, we don't need that information. Don't guess the monthly payment, always use the always use the monthly_payments tool.

[Collapse system prompt](#)

User:
How much life insurance would cost. I'm 50 years old and I want 250,000 of cover. I'd also like to pay monthly.

Assistant:
Here's the life-insurance illustration for you:

- Age: 50
- Desired coverage: \$250,000
- Annual premium: \$1,083.33
- Monthly payment: \$93.89

You can make the premium payments each month at \$93.89. Let me know if you need anything else!

Corrected Output [JSON](#)

[Click to add corrected output](#)

This prompt determines how the LLM used by the agent will approach the problem. In addition to the general instructions the prompt also includes information about the collaborators that are available to the agent.

4. Scroll past the prompt (A) to display the **User** panel (B) and the **Assistant** panel (C).

The screenshot displays the Langfuse web application interface. On the left is a sidebar with navigation options like Home, Dashboards, Tracing, Sessions, Users, Prompt Management, Prompts, Playground, Evaluation, Scores, Evaluators, Human Annotation, and Datasets. The main area is titled 'LangGraph: d67e55fd642bfa42aef9c00b8a64c1b9'. It shows a trace for an 'invoke_agent' step. The 'User' panel (B) contains the text: 'How much life insurance would cost. I'm 50 years old and I want 250,000 of cover. I'd also like to pay monthly.' The 'Assistant' panel (C) shows a tool call 'chat_with_collaborator_life_insurance' with a message: 'Please provide the annual premium for a 58-year-old customer seeking \$250,000 coverage.' A red arrow points from the Assistant panel down to the next screenshot.

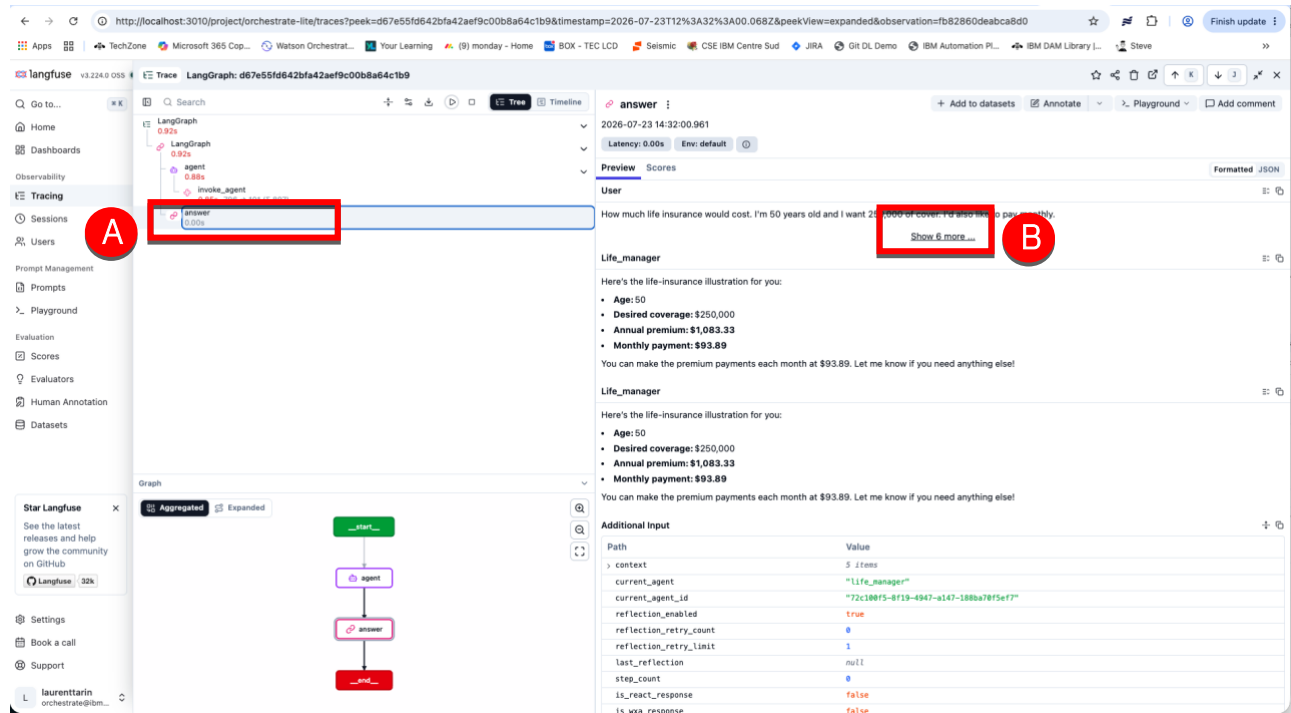
The User panel displays the text entered by the user.

5. Review the contents of the **Assistant** panel (A).

This screenshot shows the same Langfuse interface, but the Assistant panel (A) is highlighted with a red box. It displays the tool call 'chat_with_collaborator_life_insurance' and its response: 'Please provide the annual premium for a 58-year-old customer seeking \$250,000 coverage.' The 'User' panel is still visible above it.

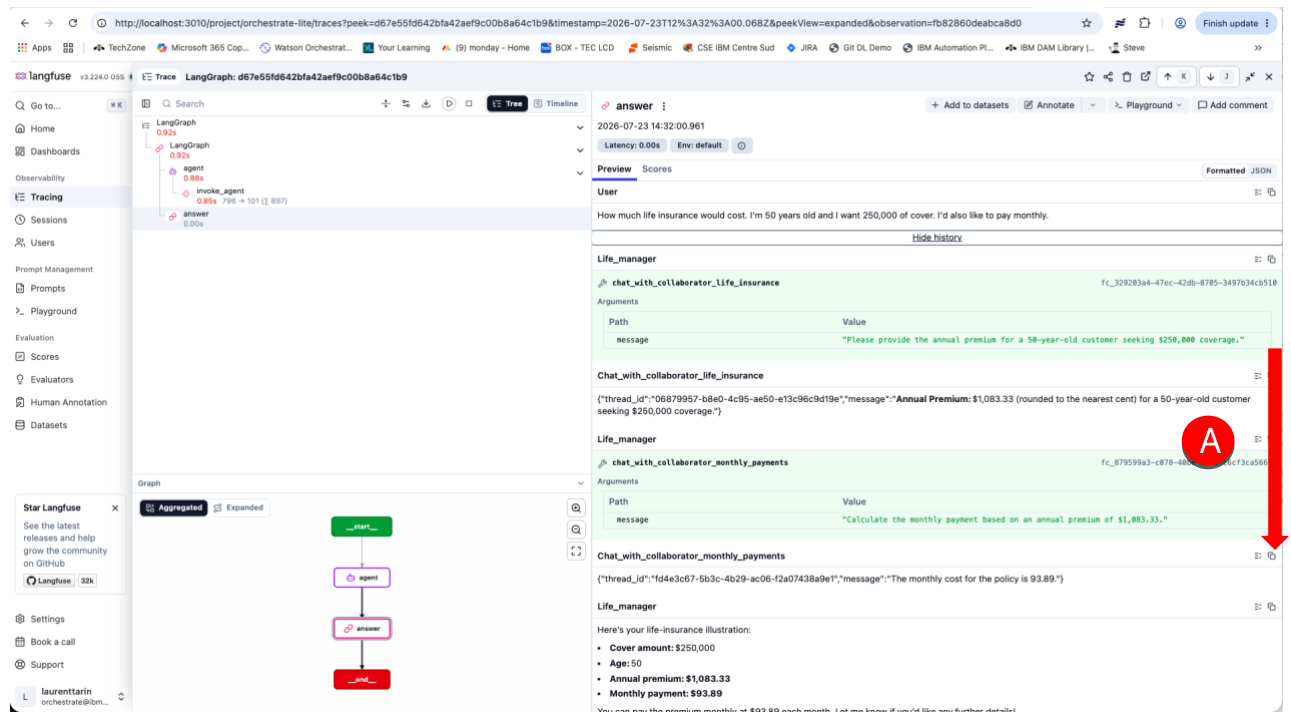
The Assistant panel contains the reasoning result from the LLM. The LLM is explaining its thought process and describing how it will proceed.

6. Click **answer** (A). Click **Show 6 more** (B).



The screenshot shows the Watsonx Orchestrator interface. On the left, there's a sidebar with navigation options like Home, Dashboards, Tracing, Sessions, Users, Prompt Management, Prompts, Playground, Evaluation, Scores, Evaluators, Human Annotation, and Datasets. The main area is divided into three panels. The top panel shows a search bar and a list of traces. The middle panel displays a graph of the workflow, with nodes for 'start', 'agent', 'answer', and 'end'. The bottom panel shows the details of the selected trace, including the 'answer' node. A red box labeled 'A' highlights the 'answer' node in the graph. Another red box labeled 'B' highlights the 'Show 6 more' button in the 'answer' node details.

7. Scroll down to and discover the Agent actions history to get to the final result (A).



The screenshot shows the Watsonx Orchestrator interface, similar to the previous one. The 'answer' node details are expanded, showing a list of actions. A red arrow points to the 'Show 6 more' button, which is labeled 'A'.

The **Life_manager** agent has first chatted with the **life_insurance** agent ("Please provide the annual premium for a 50-year-old customer seeking \$250,000 coverage.") and then with the **monthly_payment** one ("Please calculate the monthly payment for an annual premium of \$1,083.33."). Let's see how these agents have worked on their side.

- Click the Navigate up icon (A).

The screenshot shows the Langfuse interface with a trace for a **Life_manager** agent. The trace is titled "LangGraph: d67e55fd642bfa42ae9c00b8a64c1b9". The trace steps are: **start**, **agent**, **invoke_agent**, and **answer**. The **answer** step is expanded, showing a "User" request and a "Life_manager" response. The "User" request is highlighted with a red box and labeled "A". The "Life_manager" response is highlighted with a red box and labeled "B".

- You are now in the Life-insurance agent trace. You can see the user request (A) coming from the Life_manager agent, and the response to this agent (B).

The screenshot shows the Langfuse interface with a trace for a **Life_insurance** agent. The trace is titled "LangGraph: 441e8c4c-8ffe-4bb9-805a-01b3c4033e8e". The trace steps are: **start**, **agent**, **invoke_agent**, and **answer**. The **answer** step is expanded, showing a "User" request and a "Life_insurance" response. The "User" request is highlighted with a red box and labeled "A". The "Life_insurance" response is highlighted with a red box and labeled "B".

10. Click Show 2 more (A).

Langfuse v3.137.0 OSS | IBM | Watsonx Orchestrate Life Project | Traces

Trace: LangGraph: 441e8c4c-8ffe-4bb9-805a-01b3c4033e8e

Search | Timeline | LangGraph | 2026-01-21 15:10:09.663 | Session: c92da97d-eaad-40d2-9b67-b420fc601ae7 | Env: default | Latency: 10.13s

Preview | Log View | Scores | Formatted JSON

User

Please provide the annual premium for a 60-year-old customer seeking \$250,000 coverage.

Life_quote

1083.3333333333333

Life_insurance

Annual Premium: \$1,083.33 (per year) for a 60-year-old seeking \$250,000 coverage.

Additional Input

Path	Value
current_agent	"Life_insurance"
current_agent_id	"f8c1973c-94f2-4893-8c55-46692a856436"
agent_display_name	"Life_insurance"
reflection_retry_count	0
reflection_retry_limit	1
context	empty object
last_reflection	null
step_count	0
context_binary	empty object

11. Review the parameters that were transmitted to the tool

Langfuse v3.137.0 OSS | IBM | Watsonx Orchestrate Life Project | Traces

Trace: LangGraph: 441e8c4c-8ffe-4bb9-805a-01b3c4033e8e

Search | Timeline | LangGraph | 2026-01-21 15:10:09.663 | Session: c92da97d-eaad-40d2-9b67-b420fc601ae7 | Env: default | Latency: 10.13s

Preview | Log View | Scores | Formatted JSON

User

Please provide the annual premium for a 60-year-old customer seeking \$250,000 coverage.

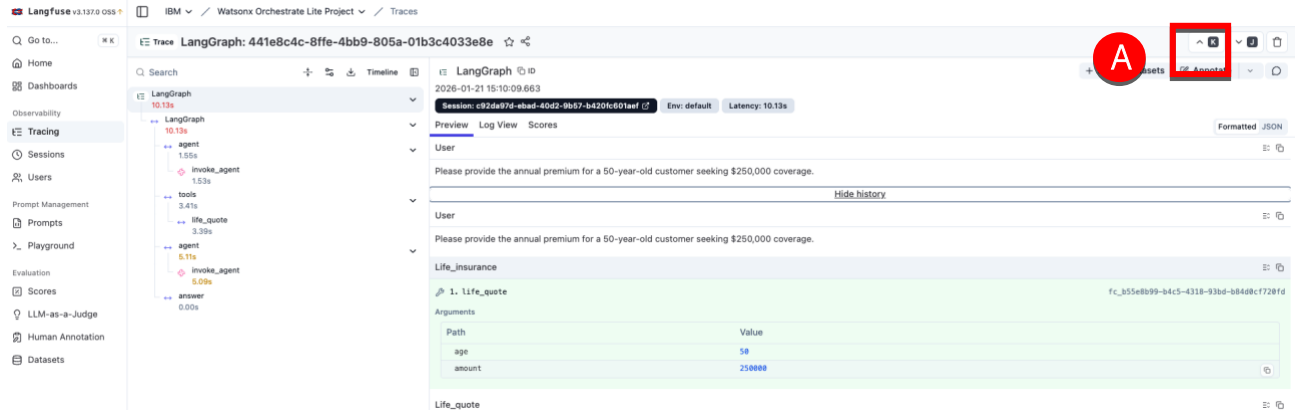
Life_insurance

Annual Premium: \$1,083.33 (per year) for a 60-year-old seeking \$250,000 coverage.

Additional Input

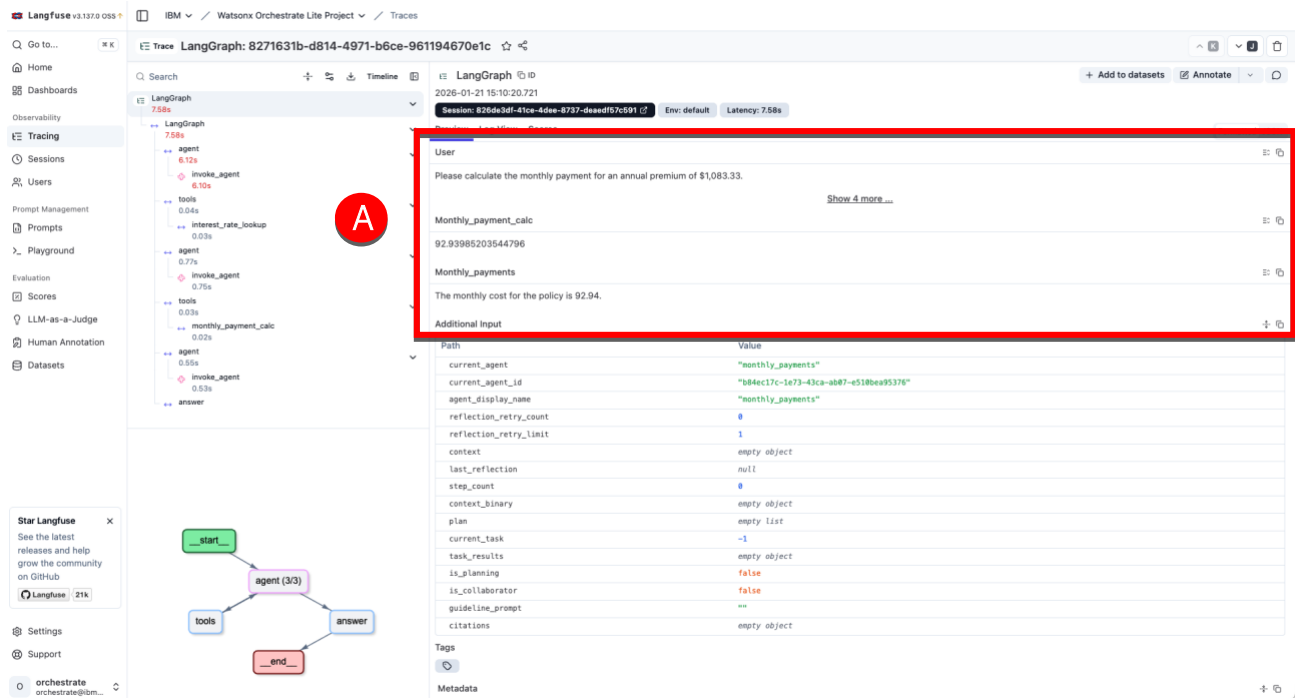
Path	Value
current_agent	"Life_insurance"
current_agent_id	"f8c1973c-94f2-4893-8c55-46692a856436"
agent_display_name	"Life_insurance"
reflection_retry_count	0
reflection_retry_limit	1
context	empty object
last_reflection	null
step_count	0
context_binary	empty object

12. Click the Navigate up icon (A).



The screenshot shows the Langfuse interface with a sidebar on the left containing navigation links like Home, Dashboards, Tracing, Sessions, Users, Prompt Management, Prompts, Playground, Evaluation, Scores, LLM-as-a-Judge, Human Annotation, and Datasets. The main area displays a trace for 'LangGraph: 441e8c4c-8ffe-4bb9-805a-01b3c4033e8e'. The trace details include a session ID, environment, latency, and a list of steps. A red circle labeled 'A' points to the 'Navigate up' icon in the top right corner of the trace view.

13. Review the trace of the Monthly_payments agent (A).



The screenshot shows the Langfuse interface with a sidebar on the left. The main area displays a trace for 'LangGraph: 8271631b-d814-4971-b6ce-961194670e1c'. The trace details include a session ID, environment, latency, and a list of steps. A red circle labeled 'A' points to the 'Monthly_payments' agent in the trace details. The trace details also show a table of arguments and a table of path values.

Path	Value
age	58
amount	250000

Summary

In this lab you created a tool from a Python function and created an agent that was able to provide quotations for life insurance. A second agent was added, able to calculate monthly payments. Finally, a third, manager agent was added that was able to combine the capabilities of the other agents.

To conclude you examined the Langfuse trace captured when running the agent.

Appendix: Troubleshooting

System unresponsive

If your ADK environment becomes unresponsive use the command below to stop all running containers.

```
docker stop $(docker ps -q)
```

Enter the following command into the terminal to reset your server, then press Enter.

```
orchestrate server reset
```

Enter the following command into the terminal to restart your server, then press Enter.

```
orchestrate server start -l -e env
```

Enter the following command into the terminal (A) to start the chat server.

```
orchestrate chat start
```

You will need to repeat any commands to create tools and agents.

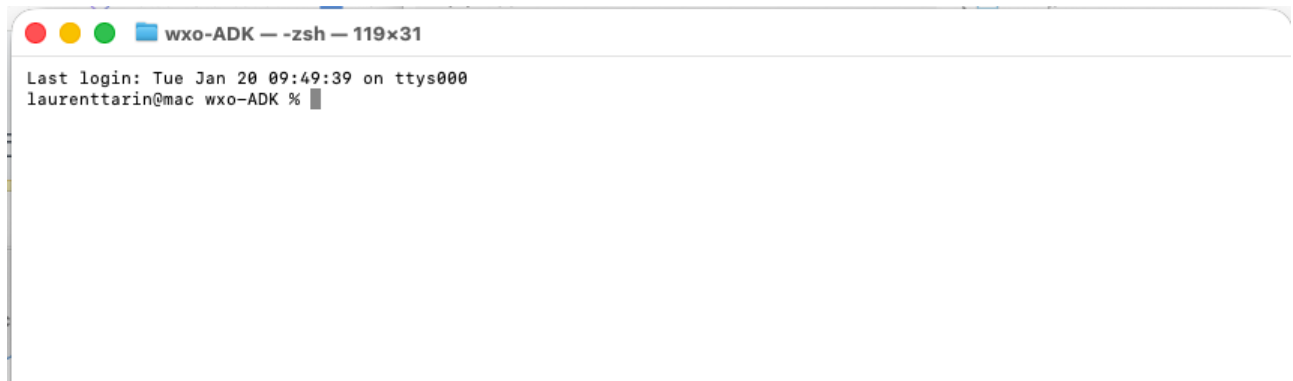
LLM Error

If you receive an error when using your agent (that worked previously), your API key may have been deleted. You will need to recreate the API key from your watsonx Orchestrate instance, update your env file, and restart the server.

Appendix: Reset your environment

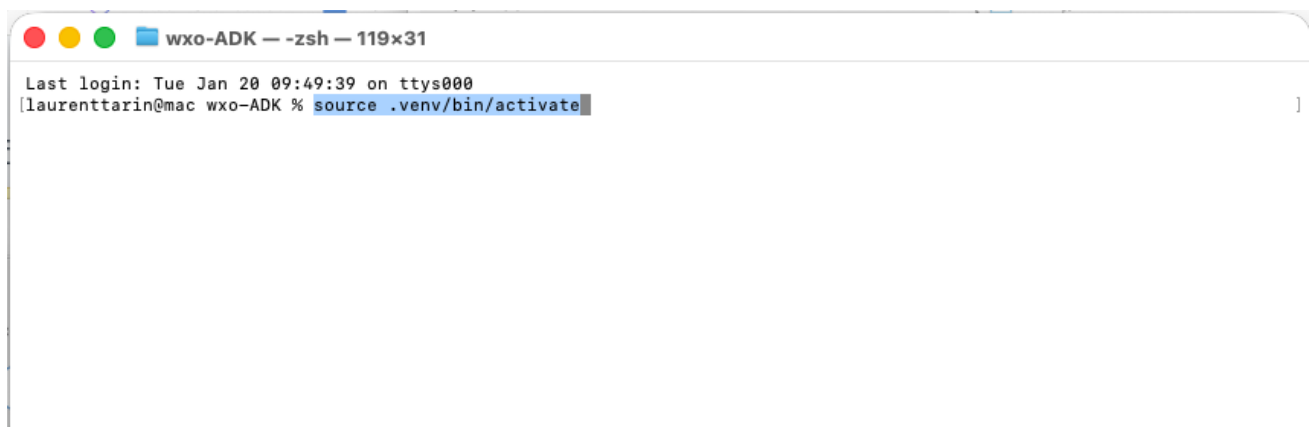
These steps will remove any existing tools and agents from your environment and restart the local watsonx Orchestrate server in readiness to start the lab.

1. Open a Terminal window on your **wxO-ADK** folder (A).



2. Enter the following command into the terminal to start your virtual environment:

```
source .venv/bin/activate
```



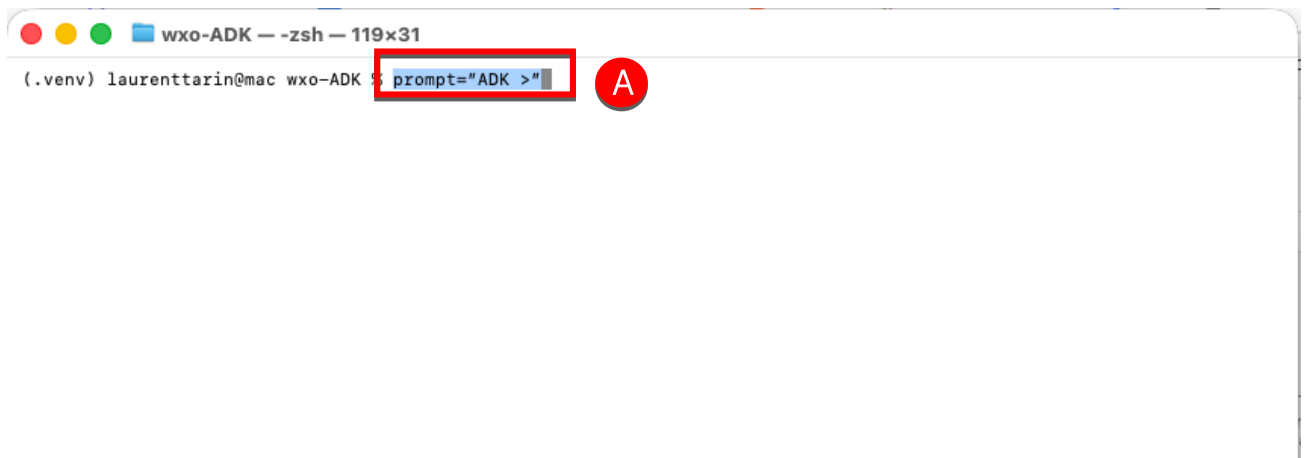
3. Check that the prompt starts with (.venv) (A) indicating that you are in your virtual environment.



```
wxo-ADK — zsh — 119x31
Last login: Tue Jan 20 09:49:39 on ttys000
[laurenttarin@mac wxo-ADK % source .venv/bin/activate
(.venv) laurenttarin@mac wxo-ADK %
```

4. Enter the following command into the terminal to improve the command visibility, then press Enter:

```
MS Windows users: function prompt{"ADK>"}
Mac users: prompt="ADK >"
```



```
(.venv) laurenttarin@mac wxo-ADK $ prompt="ADK >"
```

5. Enter the following command into the terminal to stop your chat service, then press Enter.

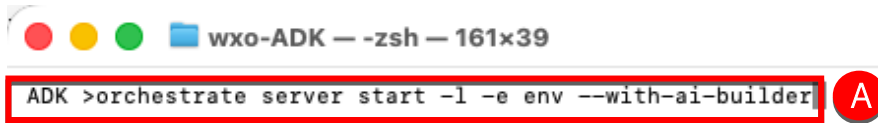
```
orchestrate chat stop
```

6. Enter the following command into the terminal to reset your server, then press Enter.

```
orchestrate server reset
```

7. Enter the following command into the terminal to start the server with your env configuration, then press Enter.

```
orchestrate server start -l -e env --with-ai-builder
```



Note: Some services may fail to start on the first attempt, and you may encounter issues connecting to the docker repository. Retry the start command in this situation, depending on the volume of requests, it can take several attempts to connect.

When startup is complete the terminal output will resemble the example below, restart can take 2-3 minutes to complete. The local server will be ready when you will see this **INFO** in the terminal screen (A):

```
wxo-ADK -- -zsh -- 119x31

Skipping already applied migration: 20251001.sql
Skipping already applied migration: 20251006.sql
Skipping already applied migration: 20251007.sql
Skipping already applied migration: 20251008.sql
Skipping already applied migration: 20251009.sql
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrator server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the
UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, pa
ssword: orchestrate
ADK >
```



You are now ready to work with watsonx Orchestrate ADK.

